

```
In [1]: import os
fname = os.path.join("datasets/jena_climate_2009_2016/jena_climate_2009_2016.csv")

with open(fname) as f:
    data = f.read()

lines = data.split("\n")
header = lines[0].split(",")
lines = lines[1:]
print(header)
print(len(lines))

['Date Time', 'p (mbar)', 'T (degC)', 'Tpot (K)', 'Tdew (degC)', 'rh (%)', 'VPmax (mbar)', 'VPact (mbar)', 'VPdef (mbar)', 'sh (g/kg)', 'H2O C (mmol/mol)', 'rho (g/m**3)', 'wv (m/s)', 'max. wv (m/s)', 'wd (deg)']
420451
```

```
In [2]: import numpy as np
temperature = np.zeros((len(lines),))
raw_data = np.zeros((len(lines), len(header) - 1))
for i, line in enumerate(lines):
    values = [float(x) for x in line.split(",")[1:]]
    temperature[i] = values[1] # Column 1 is the temperature array
    raw_data[i, :] = values[2:] # All columns (including temperature) is the "raw"
```

```
In [3]: num_train_samples = int(0.5 * len(raw_data))
num_val_samples = int(0.25 * len(raw_data))
num_test_samples = len(raw_data) - num_train_samples - num_val_samples
print("num_train_samples:", num_train_samples)
print("num_val_samples:", num_val_samples)
print("num_test_samples:", num_test_samples)
```

```
num_train_samples: 210225
num_val_samples: 105112
num_test_samples: 105114
```

```
In [4]: mean = raw_data[:num_train_samples].mean(axis=0)
raw_data -= mean
std = raw_data[:num_train_samples].std(axis=0)
raw_data /= std
```

```
In [5]: import keras

sampling_rate = 6
sequence_length = 120
delay = sampling_rate * (sequence_length + 24 - 1)
batch_size = 256

train_dataset = keras.utils.timeseries_dataset_from_array(
    raw_data[:-delay],
    targets = temperature[delay:],
    sampling_rate = sampling_rate,
    sequence_length = sequence_length,
    shuffle = True,
    batch_size = batch_size,
    start_index = 0,
    end_index = num_train_samples)
```

```

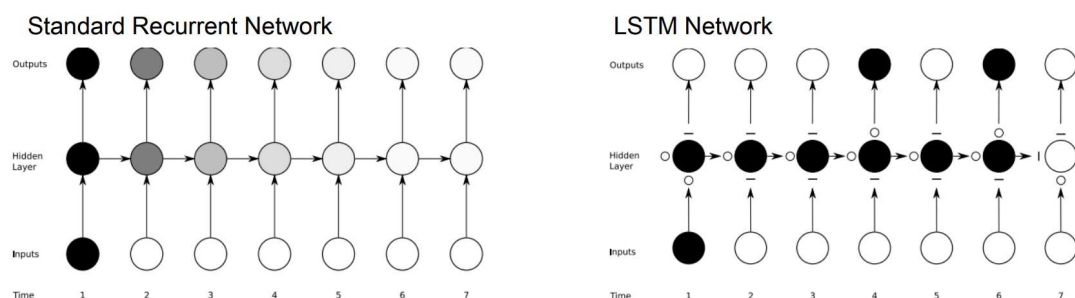
val_dataset = keras.utils.timeseries_dataset_from_array(
    raw_data[:-delay],
    targets = temperature[delay:],
    sampling_rate = sampling_rate,
    sequence_length = sequence_length,
    shuffle = True,
    batch_size = batch_size,
    start_index = num_train_samples,
    end_index = num_train_samples + num_val_samples)

test_dataset = keras.utils.timeseries_dataset_from_array(
    raw_data[:-delay],
    targets = temperature[delay:],
    sampling_rate = sampling_rate,
    sequence_length = sequence_length,
    shuffle = True,
    batch_size = batch_size,
    start_index = num_train_samples + num_val_samples)

```

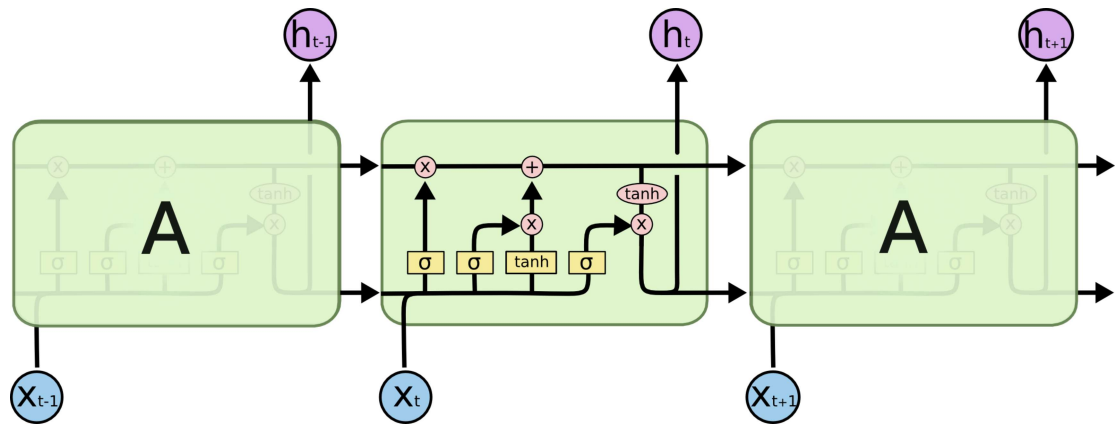
4.4. LSTM networks

- The solution to the "vanishing gradients" problem of simple RNNs is a new type of recurrent network called **Long Short-Term Memory networks** or **LSTMs**.
- LSTM networks have 'memory' of previous states of the data but this memory is selectively tuned to remember only chosen parts of past data, even for a long time.
- That is the network "remembers" past knowledge that is relevant and "forgets" irrelevant data.
- Since **the LSTM architecture aims to provide a short-term memory for RNN that can last a considerable amount of timesteps, hence the name "long short-term memory"**.
- For example, in the next image we can see that in a simple RNN the importance of a given state decays exponentially over time as new inputs overwrite the activation of the hidden unit. In a LSTM network the information can be stored for long periods of time if deemed important.

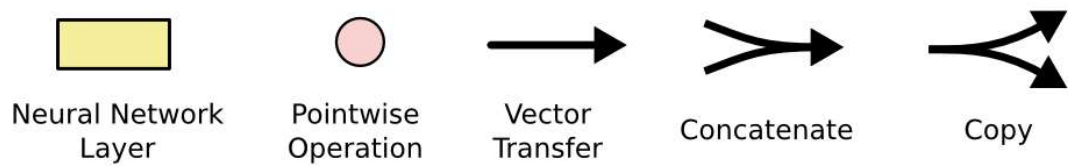


4.4.1. LSTM network structure

- LSTMs, like simple RNNs, also have a chain like structure, but the repeating module has a different structure.
- Instead of having a single neural network layer, there are four neural networks, interacting in a very special way.

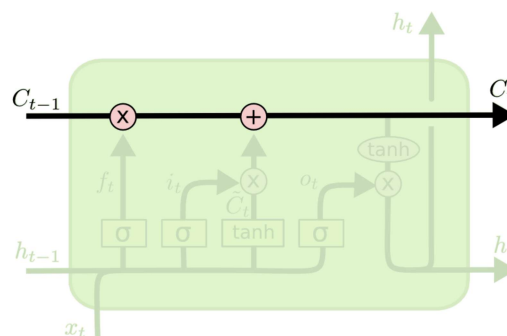


- The notation used in this graph is explained in the following figure:



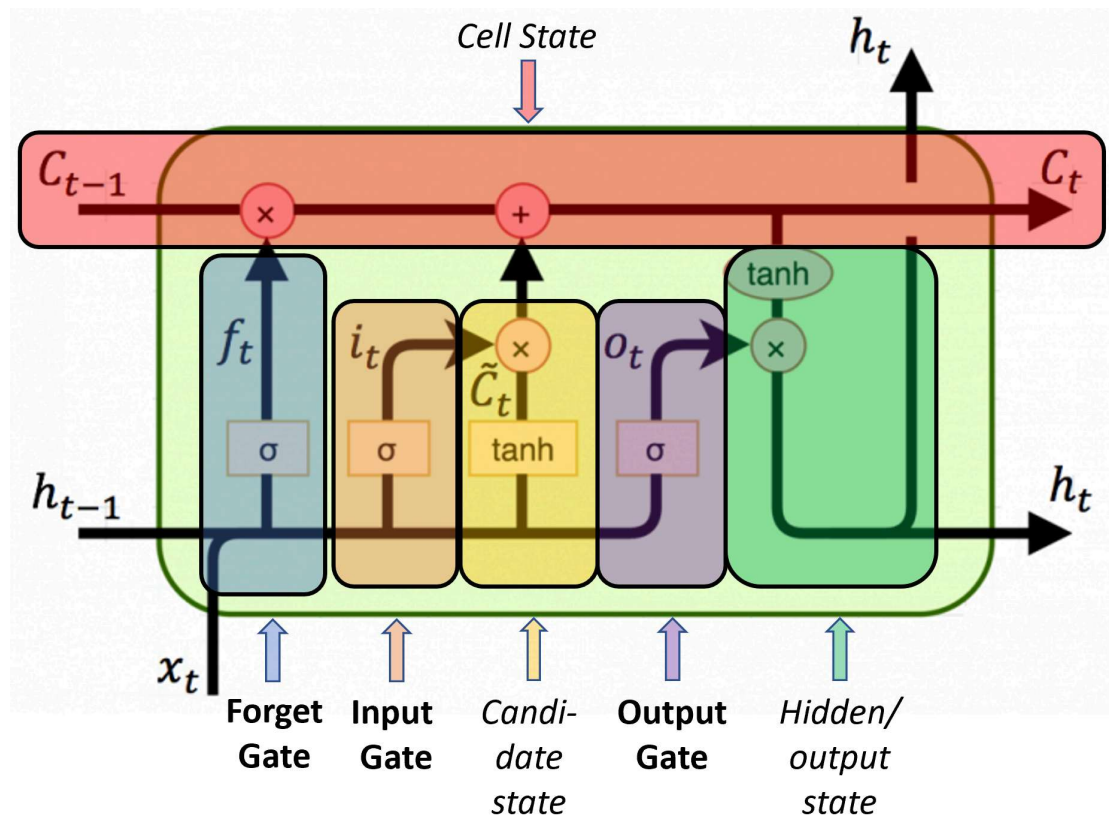
Cell state

- The cell state act as a transport highway that transfers relative information all the way down the sequence chain.
- You can think of it as the “memory” of the network.
- Therefore, information from the earlier time steps can make it’s way to later time steps, reducing the effects of short-term memory.



Gates

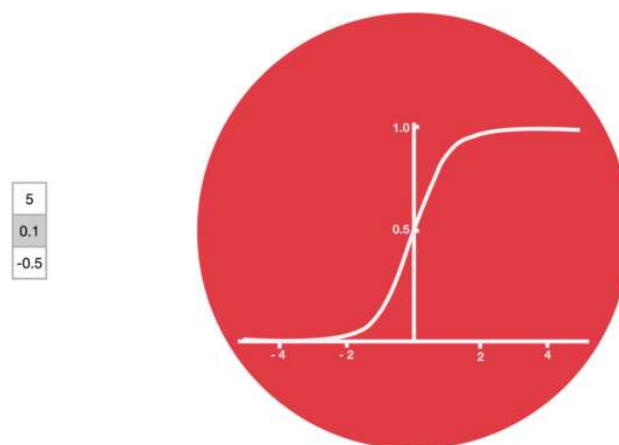
- As the cell state goes on its journey, information get’s added or removed to the cell state via gates.
- The gates are different neural networks that decide which information is allowed on the cell state.
- The gates can learn what information is relevant to keep or forget during training.
- There are three gates: the forget gate, the input gate and the output gate.



4.4.2. LSTM Walkthrough

Sigmoid

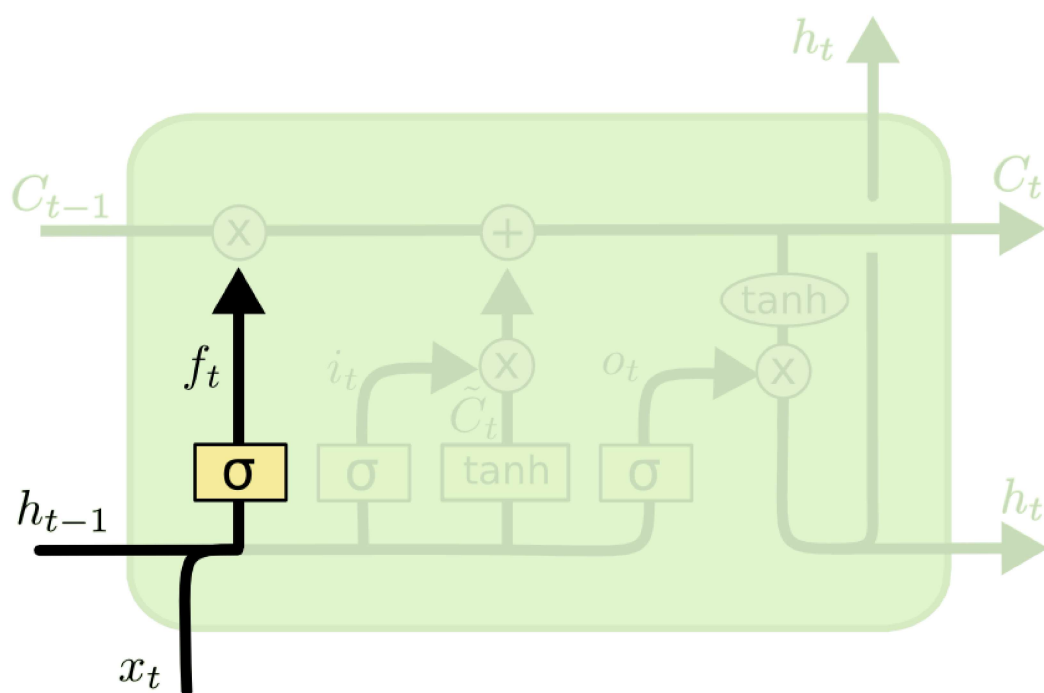
- First of all, gates contain sigmoid activations.
- A sigmoid activation is similar to the $\tanh$ activation. Instead of squishing values between -1 and 1, it squishes values between 0 and 1.
- That is helpful to update or forget data because any number getting multiplied by 0 is 0, causing values to disappear or be "forgotten."
- Any number multiplied by 1 is the same value therefore that value stay's the same or is "kept."



Forget gate

- **Objective:** Decide what information we're going to throw away from the cell state.

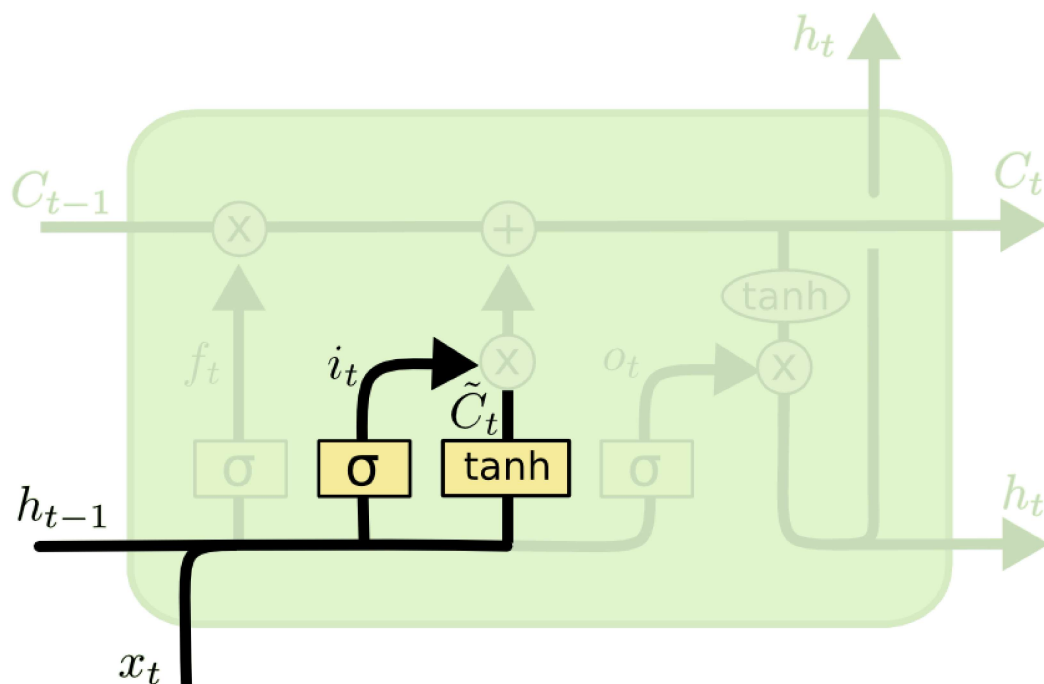
- This decision is made by a sigmoid layer called the “forget gate”.
- Information from the previous hidden state (h_{t-1}) and information from the current input (x_t) is passed through the sigmoid function.
- The gate outputs a number between 0 and 1 for each number in the cell state C_{t-1}
- A 1 represents “completely keep this” while a 0 represents “completely get rid of this.”
- **Example:** if we are using a language model trying to predict the next word based on all the previous ones. In such a problem, the cell state might include the gender of the present subject, so that the correct pronouns can be used. When we see a new subject, we want to forget the gender of the old subject.
- **Equation:** $f_t = \sigma(W_f x_t + U_f h_{t-1} + b_f)$
 - $x_t \in \mathbb{R}^d$ is the input for the current instant, with d being the dimension of the input.
 - $h_{t-1} \in (-1, 1)^h$ is the hidden state generated in instant $t - 1$, with h being the dimension of the state/output.
 - $W_f \in \mathbb{R}^{h \times d}$ is a matrix with the weights for the input.
 - $U_f \in \mathbb{R}^{h \times h}$ is a matrix with the weights for the hidden state.
 - $\sigma()$ is the sigmoid function
 - $b_f \in \mathbb{R}^h$ is the bias of the forget gate.
 - $f_t \in (0, 1)^h$ is the forget gate's activation vector.



Input gate and candidate state

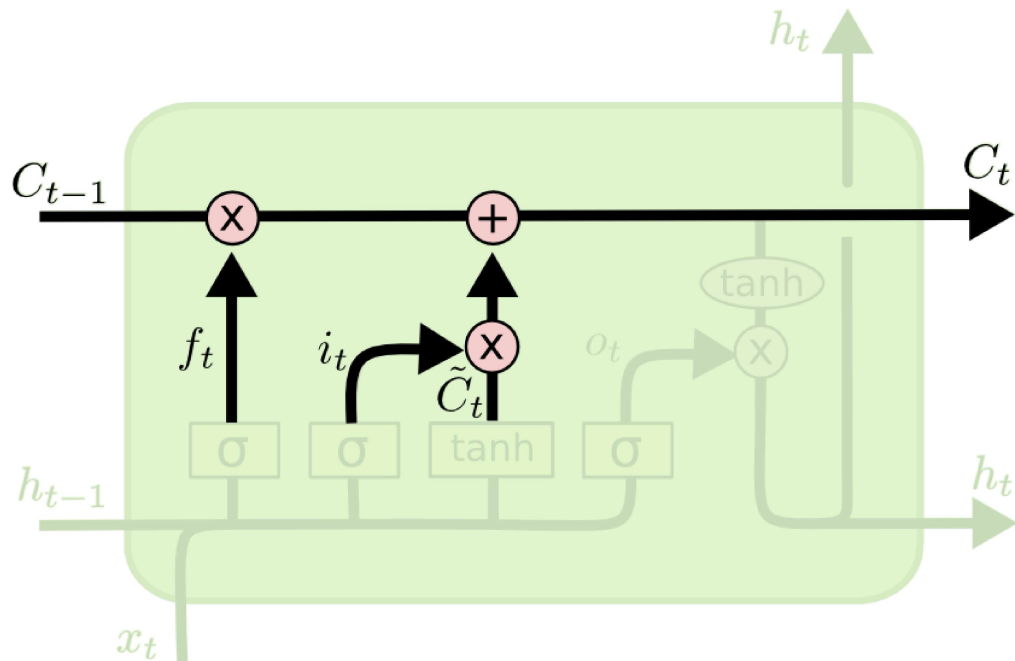
- **Objective: Decide what new information we're going to store in the cell state.**
- First part, **input gate:**
 - Decides which values we'll update.
- Second part, **candidate state:**
 - Creates a vector of new candidate values, $\tilde{C}_t$, that could be added to the state.
- In the next step, we'll combine these two to create an update to the state.

- **Example:** we'd want to add the gender of the new subject to the cell state, to replace the old one we're forgetting.
- **Equation input gate:** $i_t = \sigma(W_i x_t + U_i h_{t-1} + b_i)$
 - $i_t \in (0, 1)^h$ is the input gate's activation vector.
- **Equation candidate state:** $\tilde{C}_t = \tanh(W_c x_t + U_c h_{t-1} + b_c)$
 - $\tanh()$ is the hyperbolic tangent function.
 - $\tilde{C}_t \in (-1, 1)^h$ is the candidate state activation vector



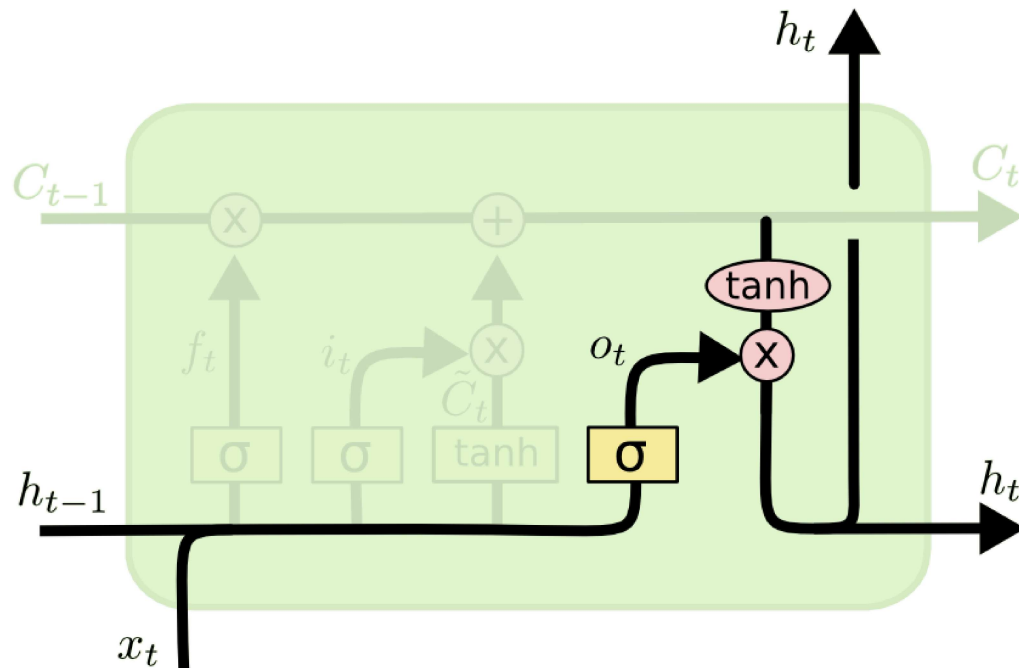
Cell state

- **Objective: Update the Cell state.**
- It's now time to update the old cell state, C_{t-1} , into the new cell state C_t . The previous steps already decided what to do, we just need to actually do it.
- We multiply the old state by f_t , forgetting the things we decided to forget earlier.
- Then we add $i_t \odot \tilde{C}_t$. This is the new candidate values, scaled by how much we decided to update each state value.
- **Example:** This is where we'd actually drop the information about the old subject's gender and add the new information, as we decided in the previous steps.
- **Equation:** $C_t = f_t \odot C_{t-1} + i_t \odot \tilde{C}_t$
 - $\odot$ is the element-wise product.
 - $C_t \in \mathbb{R}^h$ is the cell state vector



Output gate and hidden state

- **Objective: Decide what the next hidden state should be.**
 - This output will be based on our cell state, but will be a filtered version.
- First part, **output gate**:
 - We pass the previous hidden state and the current input into a sigmoid function that will decide what parts of the cell state we're going to output.
- Second part, **hidden state**:
 - We pass the newly modified cell state to the tanh function (to push the values to be between -1 and 1) and multiply it by the output of the sigmoid gate, so that we only output the parts we decided to.
- The output is the hidden state. The new cell state and the new hidden is then carried over to the next time step.
- **Example:** Since it just saw a subject, it might want to output information relevant to a verb, in case that's what is coming next. For example, it might output whether the subject is singular or plural, so that we know what form a verb should be conjugated into if that's what follows next.
- **Equation output gate:** $o_t = \sigma(W_o x_t + U_o h_{t-1} + b_o)$
 - $o_t \in (0, 1)^h$ is the output gate's activation vector.
- **Equation hidden state:** $h_t = o_t \odot \tanh(c_t)$
 - $h_t \in (-1, 1)^h$ is the hidden state vector also known as output vector of the LSTM unit.



Recap:

- The Forget gate decides what is relevant to keep from prior steps.
- The Input gate decides what information is relevant to add from the current step.
- The Output gate determines what the next hidden state should be.

4.4.3. LSTM in Keras

- In Keras the LSTM networks are created using the `LSTM` layer (https://keras.io/api/layers/recurrent_layers/lstm/)
- The main parameter to use is `units`, it is a positive integer that represents the dimensionality of the output space.

```
In [6]: import keras
from keras import layers

inputs = keras.Input(shape=(sequence_length, raw_data.shape[-1]))
x = layers.LSTM(16)(inputs)
outputs = layers.Dense(1)(x)
model = keras.Model(inputs, outputs)

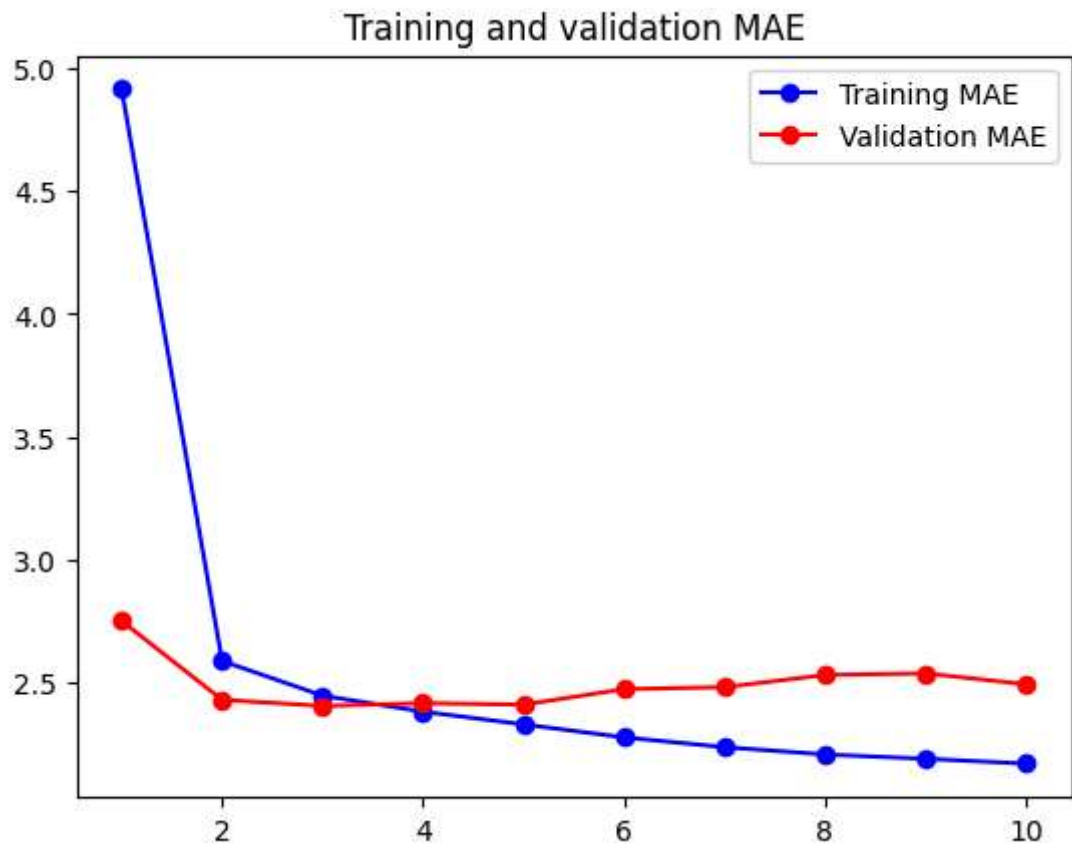
callbacks = [
    keras.callbacks.ModelCheckpoint("jena_lstm.keras",
                                    save_best_only=True)
]
model.compile(optimizer="rmsprop", loss="mse", metrics=["mae"])
history = model.fit(train_dataset,
                    epochs=10,
                    validation_data=val_dataset,
                    callbacks=callbacks)
model = keras.models.load_model("jena_lstm.keras")
print(f"Test MAE: {model.evaluate(test_dataset)[1]:.2f}")
```

Epoch 1/10
819/819 ————— **30s** 35ms/step - loss: 78.0151 - mae: 6.7838 - val_loss: 13.2045 - val_mae: 2.7504
Epoch 2/10
819/819 ————— **26s** 32ms/step - loss: 12.0644 - mae: 2.6770 - val_loss: 9.7766 - val_mae: 2.4296
Epoch 3/10
819/819 ————— **27s** 33ms/step - loss: 9.9551 - mae: 2.4592 - val_loss: 9.5658 - val_mae: 2.4042
Epoch 4/10
819/819 ————— **28s** 34ms/step - loss: 9.4474 - mae: 2.3929 - val_loss: 9.6121 - val_mae: 2.4158
Epoch 5/10
819/819 ————— **27s** 33ms/step - loss: 9.0296 - mae: 2.3382 - val_loss: 9.5186 - val_mae: 2.4098
Epoch 6/10
819/819 ————— **25s** 31ms/step - loss: 8.6150 - mae: 2.2847 - val_loss: 10.1198 - val_mae: 2.4729
Epoch 7/10
819/819 ————— **25s** 30ms/step - loss: 8.2530 - mae: 2.2411 - val_loss: 10.1231 - val_mae: 2.4810
Epoch 8/10
819/819 ————— **24s** 30ms/step - loss: 7.9915 - mae: 2.2073 - val_loss: 10.5024 - val_mae: 2.5304
Epoch 9/10
819/819 ————— **24s** 29ms/step - loss: 7.8131 - mae: 2.1857 - val_loss: 10.5454 - val_mae: 2.5371
Epoch 10/10
819/819 ————— **25s** 30ms/step - loss: 7.6831 - mae: 2.1706 - val_loss: 10.1655 - val_mae: 2.4928
405/405 ————— **5s** 12ms/step - loss: 10.2208 - mae: 2.4873
Test MAE: 2.49

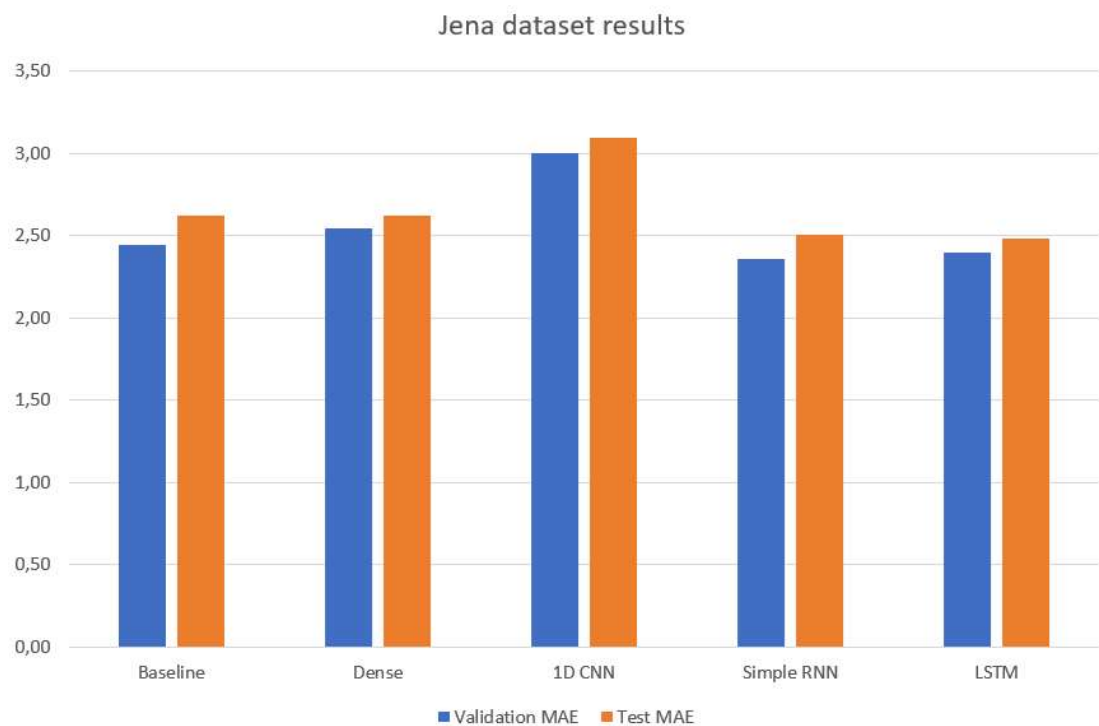
In [7]: `import matplotlib.pyplot as plt`

```
def plot_mae(history):
    loss = history.history["mae"]
    val_loss = history.history["val_mae"]
    epochs = range(1, len(loss) + 1)
    plt.figure()
    plt.plot(epochs, loss, "b-o", label="Training MAE")
    plt.plot(epochs, val_loss, "r-o", label="Validation MAE")
    plt.title("Training and validation MAE")
    plt.legend()
    plt.show()
```

In [8]: `plot_mae(history)`

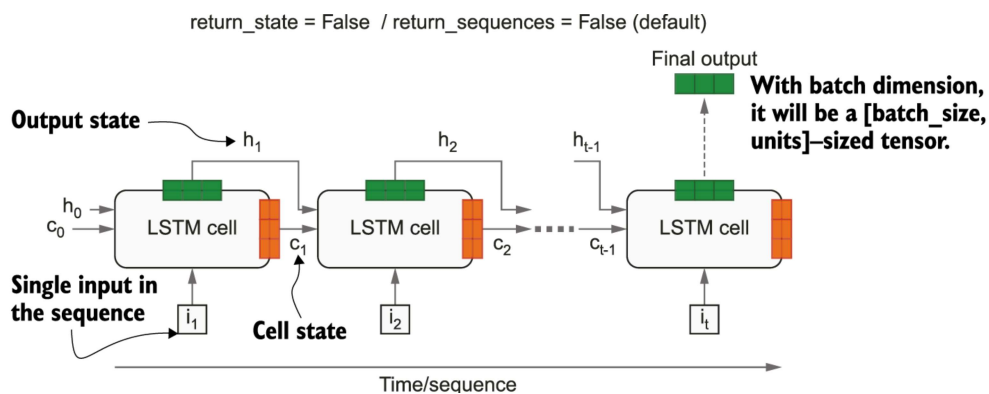


- We beat again the common-sense baseline.
- The results are similar to the Simple RNN.
- The network shows signs of overfitting (we will fix this later).

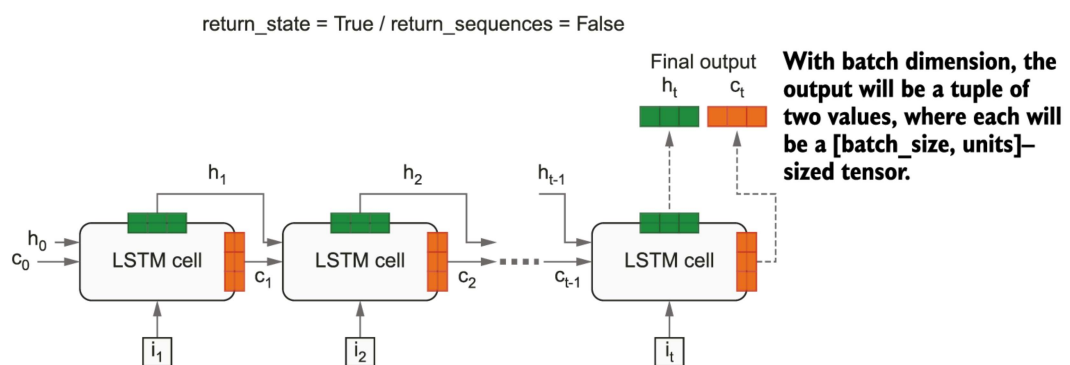


`return_state` and `return_sequences` parameters

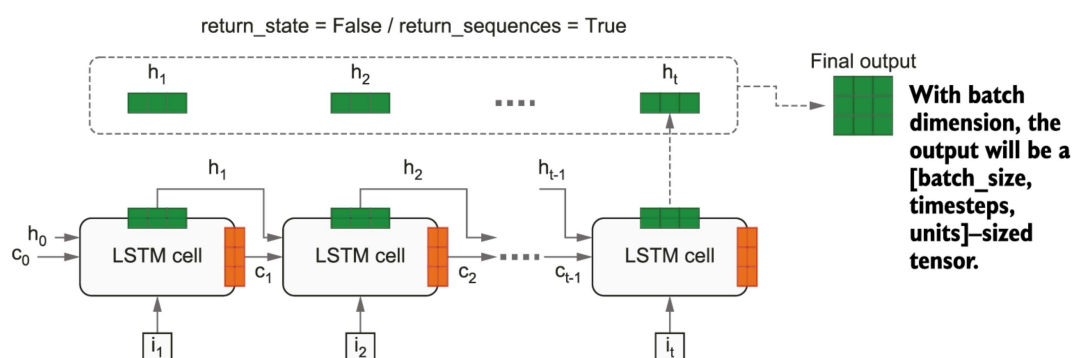
- The default option is `return_state` and `return_sequences` to be set as `False`.
- The network returns the final hidden state as output.



- If `return_state` is set to `True` the final cell state is also returned.



- If `return_sequences` is set to `True` the hidden state of each timestep is returned.

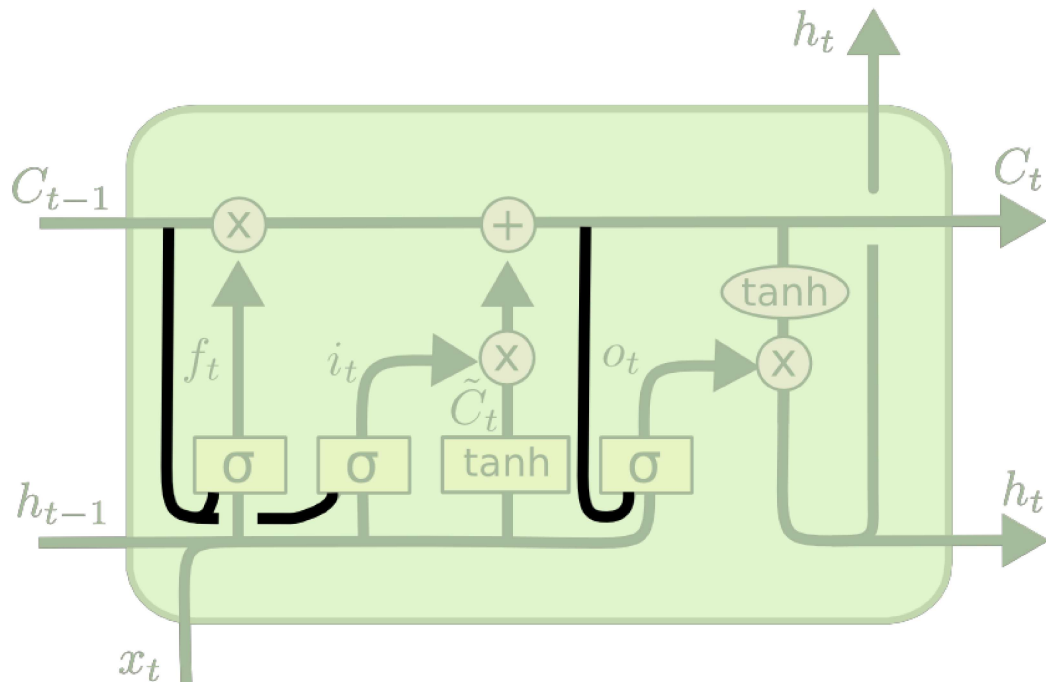


4.4.4. LSTM variations

Peephole connections

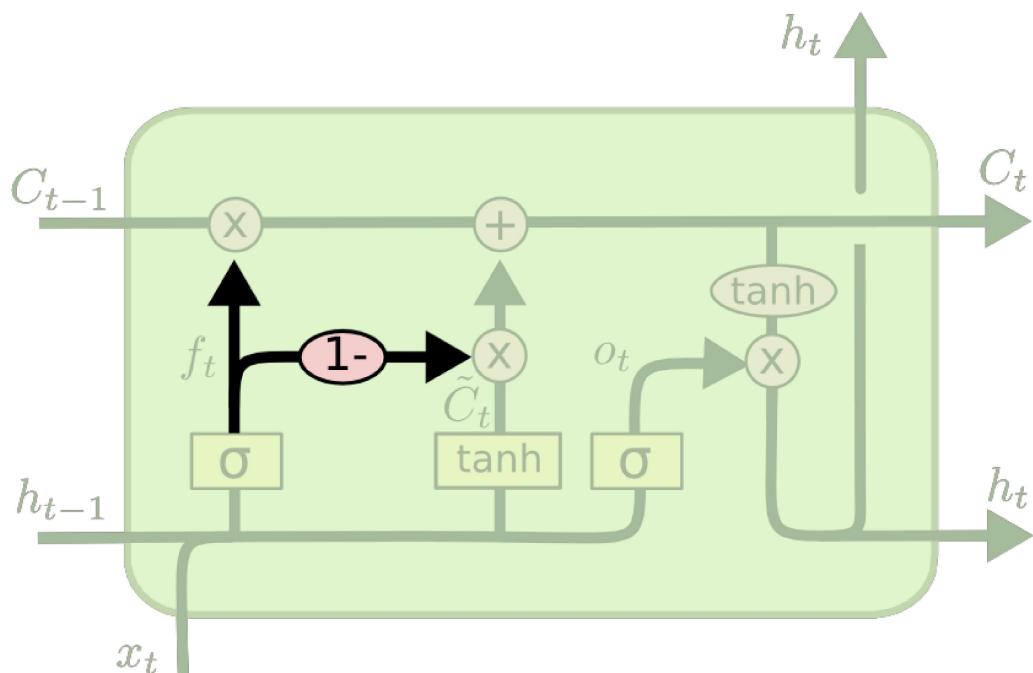
- We let the gate layers look at the cell state.
- The diagram adds peepholes to all the gates, but there are variations that will give some peepholes and not others.
- **Equations:**

- $f_t = \sigma(W_f x_t + U_f h_{t-1} + V_f C_{t-1} + b_f)$
- $i_t = \sigma(W_i x_t + U_i h_{t-1} + V_i C_{t-1} + b_i)$
- $o_t = \sigma(W_o x_t + U_o h_{t-1} + V_o C_{t-1} + b_o)$
- $V_{f,i,o} \in \mathbb{R}^{h \times h}$ are matrices with the weights for the cell state.



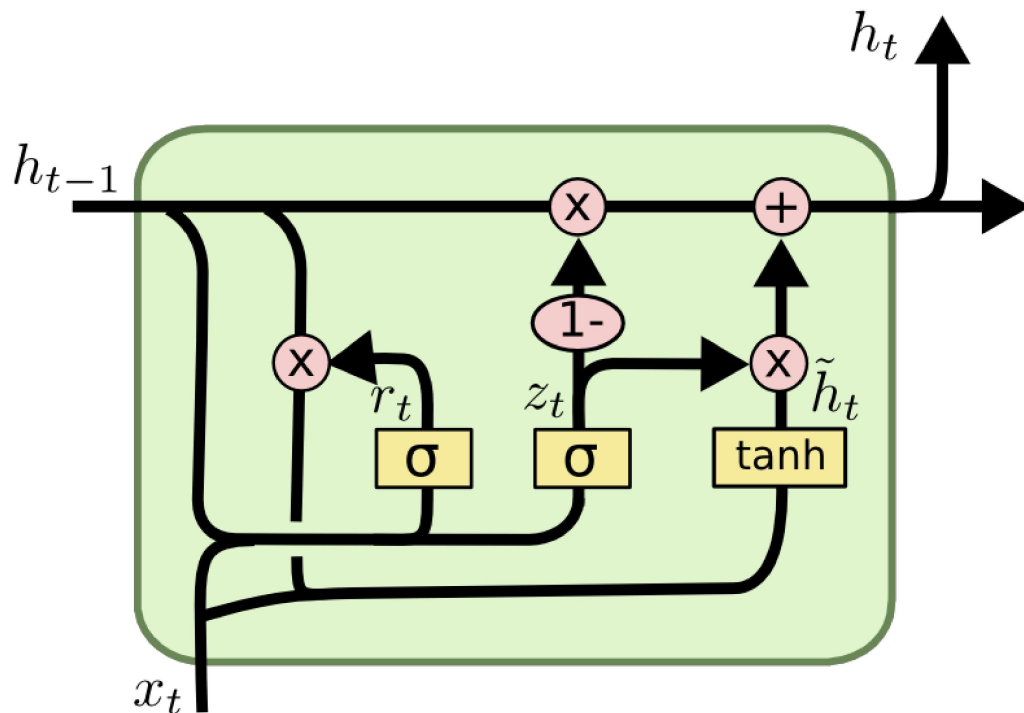
Coupled forget and input gates

- Instead of separately deciding what to forget and what we should add new information to, we make those decisions together.
- We only forget when we're going to input something in its place.
- We only input new values to the state when we forget something older.
- **Equation:** $C_t = f_t \odot C_{t-1} + (1 - f_t) \odot \tilde{C}_t$

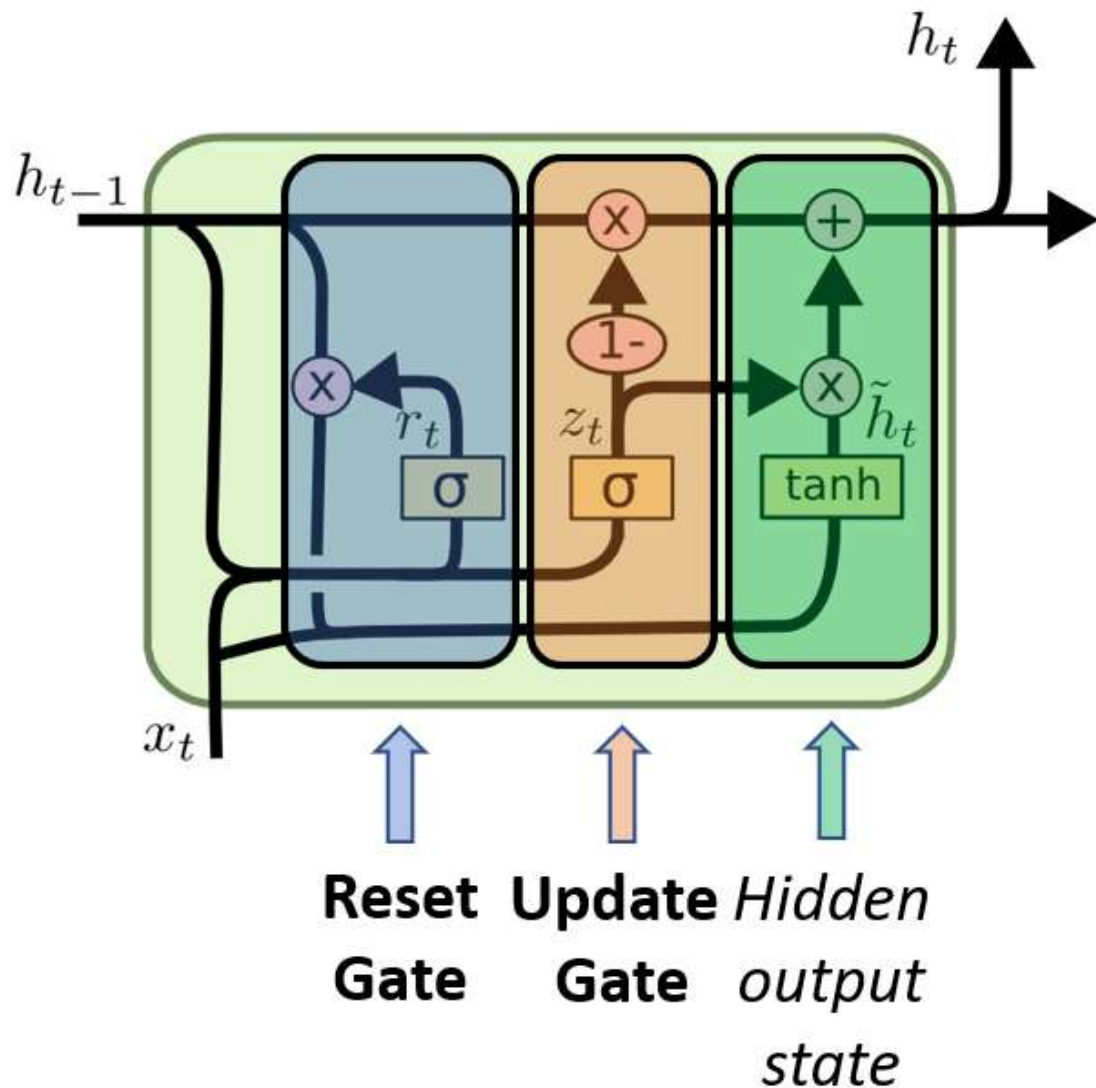


4.5. GRU networks

- The GRU (Gated Recurrent Unit) is the newer generation of Recurrent Neural networks and is pretty similar to an LSTM but simpler.



- **Changes:**
 - It merges the cell state and hidden state to transfer information.
 - It combines the forget and input gates into a single **update gate** that decides what information to throw away and what new information to add. In a similar way to coupled forget and input gates.
 - Adds a **reset gate** to decide how much past information to forget.



- **Equations:**

- update gate: $z_t = \sigma(W_z x_t + U_z h_{t-1} + b_z)$
- reset gate: $r_t = \sigma(W_r x_t + U_r h_{t-1} + b_r)$
- hidden state:
 - $\tilde{h}_t = \tanh(W_h x_t + U_h(r_t \odot h_{t-1}) + b_h)$
 - $h_t = (1 - z_t) \odot h_{t-1} + z_t \odot \tilde{h}_t$

4.5.1 LSTM vs. GRU

- GRU uses less training parameter and therefore uses less memory and executes faster than LSTM.
- Generally, GRUs outperform LSTM networks on low-complexity sequences while on high-complexity sequences LSTMs perform better.
- In conclusion, one can choose LSTM if you are dealing with large sequences and accuracy is concerned, GRU is used when you have less memory consumption and want faster results.

4.5.2. GRUs in Keras

- We can easily build GRU networks in Keras using the `GRU` layer (https://keras.io/api/layers/recurrent_layers/gru/).

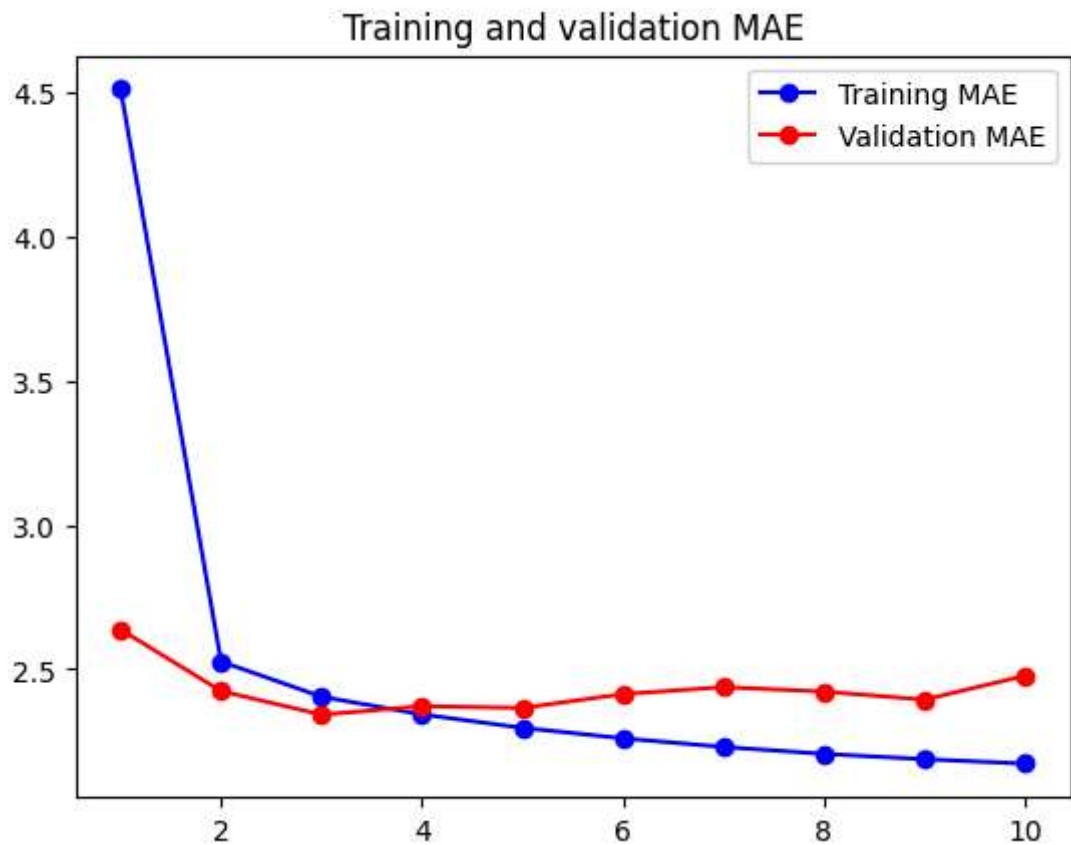
```
In [9]: from tensorflow import keras
        from tensorflow.keras import layers

        inputs = keras.Input(shape=(sequence_length, raw_data.shape[-1]))
        x = layers.GRU(16)(inputs)
        outputs = layers.Dense(1)(x)
        model = keras.Model(inputs, outputs)

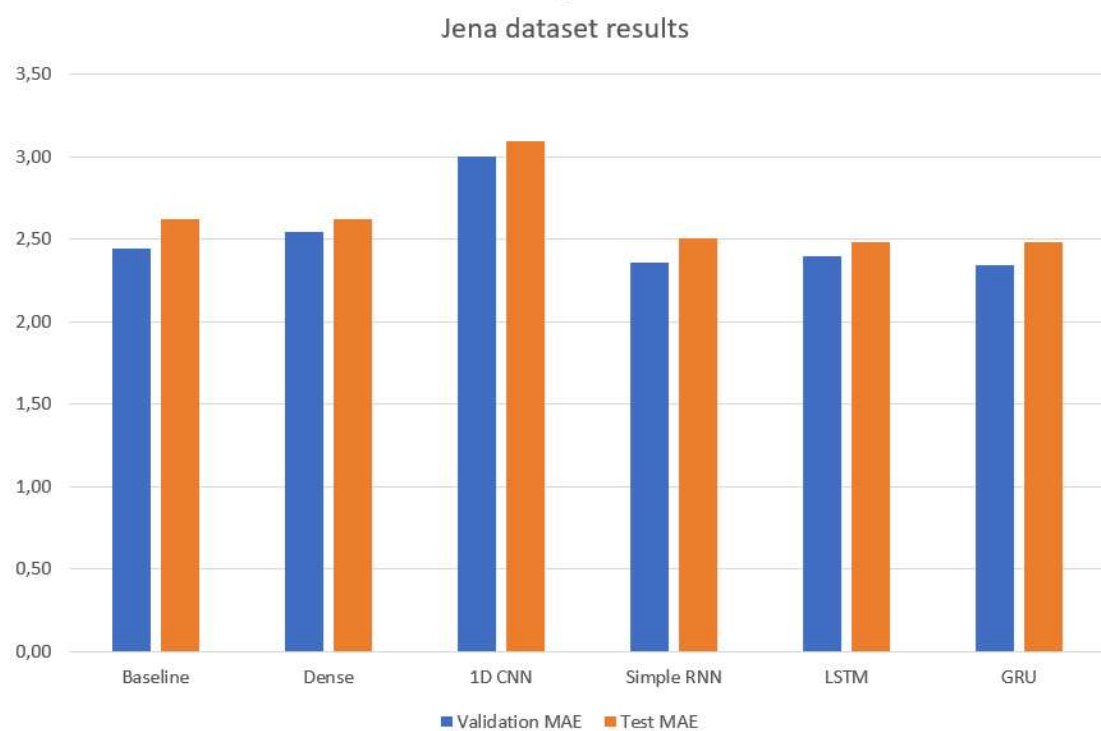
        callbacks = [
            keras.callbacks.ModelCheckpoint("jena_gru.keras",
                                           save_best_only=True)
        ]
        model.compile(optimizer="rmsprop", loss="mse", metrics=["mae"])
        history = model.fit(train_dataset,
                            epochs=10,
                            validation_data=val_dataset,
                            callbacks=callbacks)
        model = keras.models.load_model("jena_gru.keras")
        print(f"Test MAE: {model.evaluate(test_dataset)[1]:.2f}")
```

```
Epoch 1/10
819/819 ————— 28s 33ms/step - loss: 66.9168 - mae: 6.2439 - val_loss: 12.2925 - val_mae: 2.6372
Epoch 2/10
819/819 ————— 27s 33ms/step - loss: 11.2056 - mae: 2.5909 - val_loss: 10.0088 - val_mae: 2.4237
Epoch 3/10
819/819 ————— 27s 33ms/step - loss: 9.6400 - mae: 2.4208 - val_loss: 9.2566 - val_mae: 2.3421
Epoch 4/10
819/819 ————— 54s 66ms/step - loss: 9.1433 - mae: 2.3523 - val_loss: 9.6297 - val_mae: 2.3716
Epoch 5/10
819/819 ————— 27s 33ms/step - loss: 8.7554 - mae: 2.3033 - val_loss: 9.5352 - val_mae: 2.3655
Epoch 6/10
819/819 ————— 27s 33ms/step - loss: 8.4623 - mae: 2.2669 - val_loss: 10.0404 - val_mae: 2.4135
Epoch 7/10
819/819 ————— 26s 32ms/step - loss: 8.2121 - mae: 2.2357 - val_loss: 10.3517 - val_mae: 2.4376
Epoch 8/10
819/819 ————— 26s 32ms/step - loss: 8.0199 - mae: 2.2103 - val_loss: 10.1099 - val_mae: 2.4221
Epoch 9/10
819/819 ————— 26s 32ms/step - loss: 7.8628 - mae: 2.1903 - val_loss: 9.7611 - val_mae: 2.3942
Epoch 10/10
819/819 ————— 26s 32ms/step - loss: 7.7437 - mae: 2.1748 - val_loss: 10.7629 - val_mae: 2.4770
405/405 ————— 4s 10ms/step - loss: 10.0821 - mae: 2.4881
Test MAE: 2.49
```

```
In [10]: plot_mae(history)
```



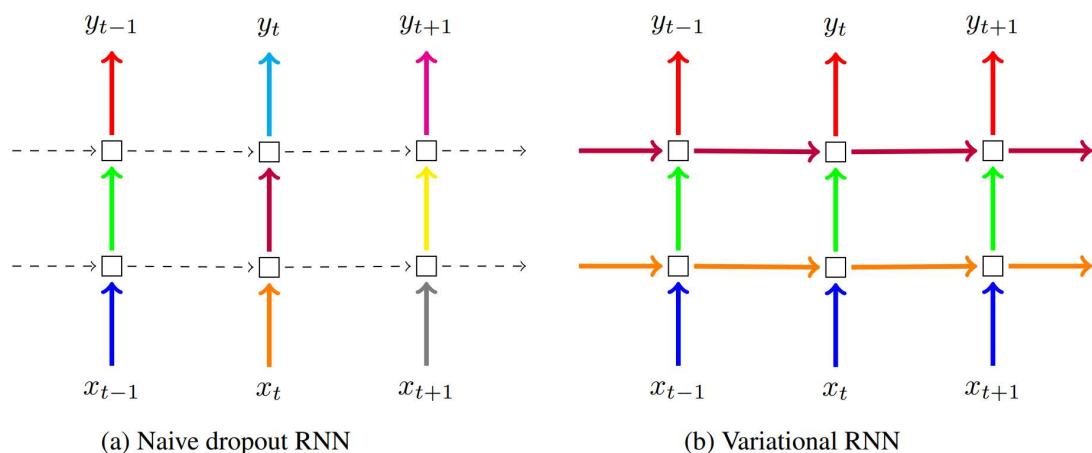
- GRU's has fewer tensor operations; therefore, they are a little speedier to train than LSTM's.
- Results are similar to LSTMs and there isn't a clear winner which one is better.



4.6. Advanced use of RNNs

4.6.1. Recurrent dropout

- The results of LSTM and GRU show some signs of overfitting, so we may need to use dropout.
- Dropout was originally applied to the inputs of the recurrent units and varying randomly from timestep to timestep (option a in the figure).
- It was determined that the proper way to use dropout with a recurrent network is to **apply the same dropout mask (the same pattern of dropped units) at every timestep**, instead of using a dropout mask that varies randomly from timestep to timestep (see <https://arxiv.org/pdf/1512.05287.pdf>).
- Also a constant dropout mask should be **applied to the inner recurrent activations of the layer** (option b in the figure).



- Every recurrent layer in Keras has **two dropout-related arguments**:
 - `dropout` : a float specifying the dropout rate for input units of the layer.
 - `recurrent_dropout` : specifying the dropout rate of the recurrent units.

Example:

- Let's add recurrent dropout to the LSTM layer of our first LSTM example and see how doing so impacts overfitting.
- We are only apply recurrent dropout (normally applying dropout before a recurrent layer hinders learning rather than helping with regularization).
- To regularize the Dense layer, we also add a Dropout layer after the LSTM.

```
In [11]: inputs = keras.Input(shape=(sequence_length, raw_data.shape[-1]))
x = layers.LSTM(32, recurrent_dropout=0.25)(inputs)
x = layers.Dropout(0.5)(x)
outputs = layers.Dense(1)(x)
model = keras.Model(inputs, outputs)

callbacks = [
    keras.callbacks.ModelCheckpoint("jena_lstm_dropout.keras",
                                    save_best_only=True)
]
model.compile(optimizer="rmsprop", loss="mse", metrics=["mae"])
history = model.fit(train_dataset,
                    epochs=20,
```

```
        validation_data=val_dataset,  
        callbacks=callbacks)  
model = keras.models.load_model("jena_lstm_dropout.keras")  
print(f"Test MAE: {model.evaluate(test_dataset)[1]:.2f}")
```

Epoch 1/20
819/819  **45s** 54ms/step - loss: 49.9034 - mae: 5.2366 - val_loss: 9.7261 - val_mae: 2.4264

Epoch 2/20
819/819  **40s** 49ms/step - loss: 15.2596 - mae: 3.0380 - val_loss: 9.2573 - val_mae: 2.3698

Epoch 3/20
819/819  **41s** 50ms/step - loss: 14.5015 - mae: 2.9513 - val_loss: 9.2505 - val_mae: 2.3718

Epoch 4/20
819/819  **35s** 43ms/step - loss: 13.9424 - mae: 2.8907 - val_loss: 9.3877 - val_mae: 2.3828

Epoch 5/20
819/819  **33s** 40ms/step - loss: 13.4727 - mae: 2.8478 - val_loss: 9.2497 - val_mae: 2.3553

Epoch 6/20
819/819  **33s** 41ms/step - loss: 13.2028 - mae: 2.8170 - val_loss: 9.1717 - val_mae: 2.3447

Epoch 7/20
819/819  **34s** 42ms/step - loss: 12.8612 - mae: 2.7817 - val_loss: 8.9086 - val_mae: 2.3169

Epoch 8/20
819/819  **34s** 42ms/step - loss: 12.4735 - mae: 2.7413 - val_loss: 9.0486 - val_mae: 2.3302

Epoch 9/20
819/819  **34s** 41ms/step - loss: 12.3203 - mae: 2.7264 - val_loss: 8.8149 - val_mae: 2.3007

Epoch 10/20
819/819  **35s** 42ms/step - loss: 12.1746 - mae: 2.7070 - val_loss: 8.9780 - val_mae: 2.3215

Epoch 11/20
819/819  **35s** 42ms/step - loss: 12.0569 - mae: 2.6972 - val_loss: 9.2034 - val_mae: 2.3485

Epoch 12/20
819/819  **36s** 44ms/step - loss: 11.8247 - mae: 2.6679 - val_loss: 8.9629 - val_mae: 2.3181

Epoch 13/20
819/819  **36s** 44ms/step - loss: 11.7433 - mae: 2.6597 - val_loss: 9.0173 - val_mae: 2.3287

Epoch 14/20
819/819  **36s** 44ms/step - loss: 11.6986 - mae: 2.6558 - val_loss: 8.9692 - val_mae: 2.3245

Epoch 15/20
819/819  **36s** 44ms/step - loss: 11.5478 - mae: 2.6403 - val_loss: 8.8691 - val_mae: 2.3132

Epoch 16/20
819/819  **36s** 44ms/step - loss: 11.5594 - mae: 2.6395 - val_loss: 8.9783 - val_mae: 2.3252

Epoch 17/20
819/819  **36s** 44ms/step - loss: 11.4372 - mae: 2.6284 - val_loss: 9.1075 - val_mae: 2.3466

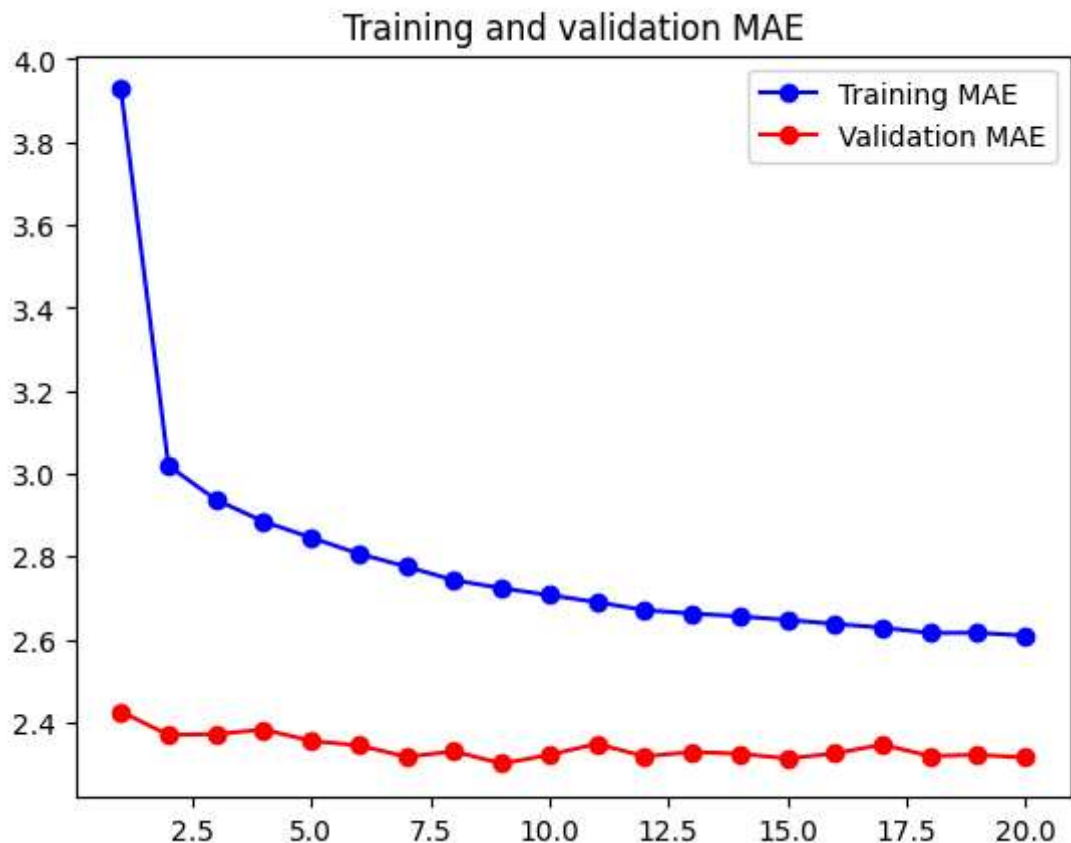
Epoch 18/20
819/819  **36s** 44ms/step - loss: 11.3086 - mae: 2.6153 - val_loss: 8.9460 - val_mae: 2.3188

Epoch 19/20
819/819  **36s** 44ms/step - loss: 11.3262 - mae: 2.6132 - val_loss: 8.9751 - val_mae: 2.3220

Epoch 20/20
819/819  **36s** 44ms/step - loss: 11.2118 - mae: 2.6042 - val_loss: 8.9039 - val_mae: 2.3153

405/405 — 5s 12ms/step - loss: 9.8904 - mae: 2.4885
Test MAE: 2.49

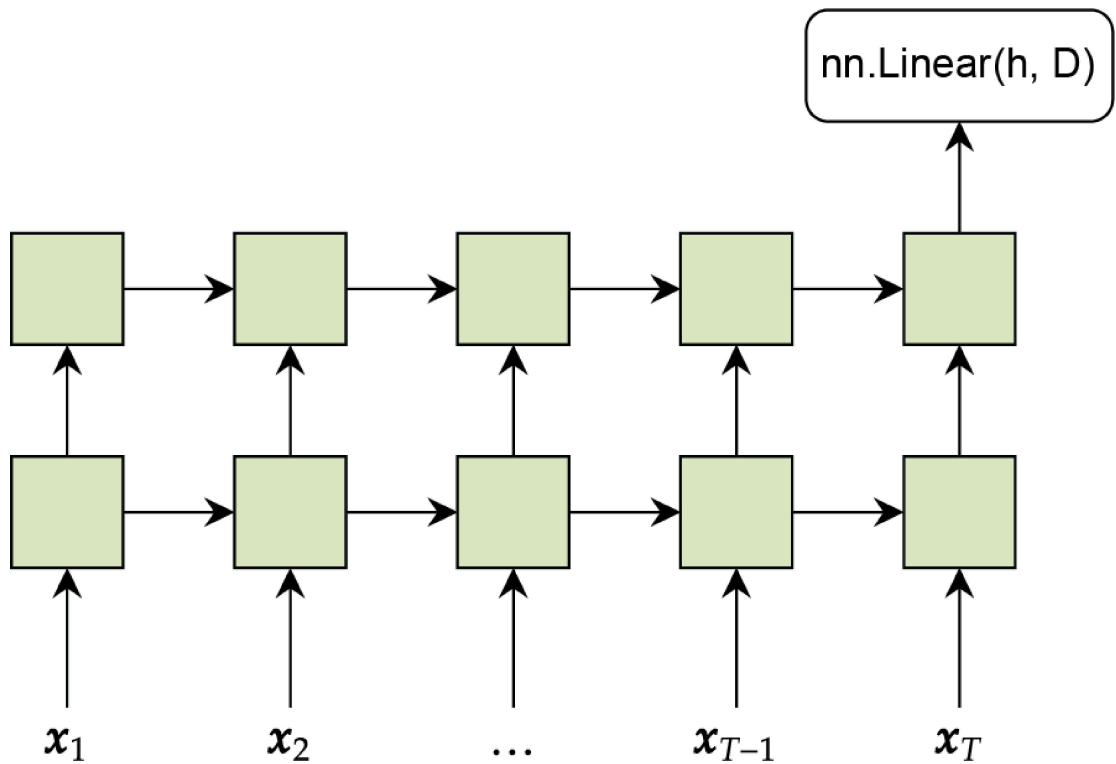
```
In [12]: plot_mae(history)
```



The results are similar to other recurrent versions, but the network takes longer to start overfitting.

4.6.2. Stacking recurrent layers

- After regularizing our recurrent network we can consider increasing the capacity and expressive power of the network.
- Increasing network capacity is typically done by **increasing the number of units in the layers** or **adding more layers**.
- To stack recurrent layers on top of each other in Keras, all intermediate layers should return their full sequence of outputs (`return_sequences=True`) rather than their output at the last timestep.
- Several layers usually perform better, but are more expensive to train.



```
In [13]: inputs = keras.Input(shape=(sequence_length, raw_data.shape[-1]))
x = layers.GRU(32, recurrent_dropout=0.5, return_sequences=True)(inputs)
x = layers.GRU(32, recurrent_dropout=0.5)(x)
x = layers.Dropout(0.5)(x)
outputs = layers.Dense(1)(x)
model = keras.Model(inputs, outputs)

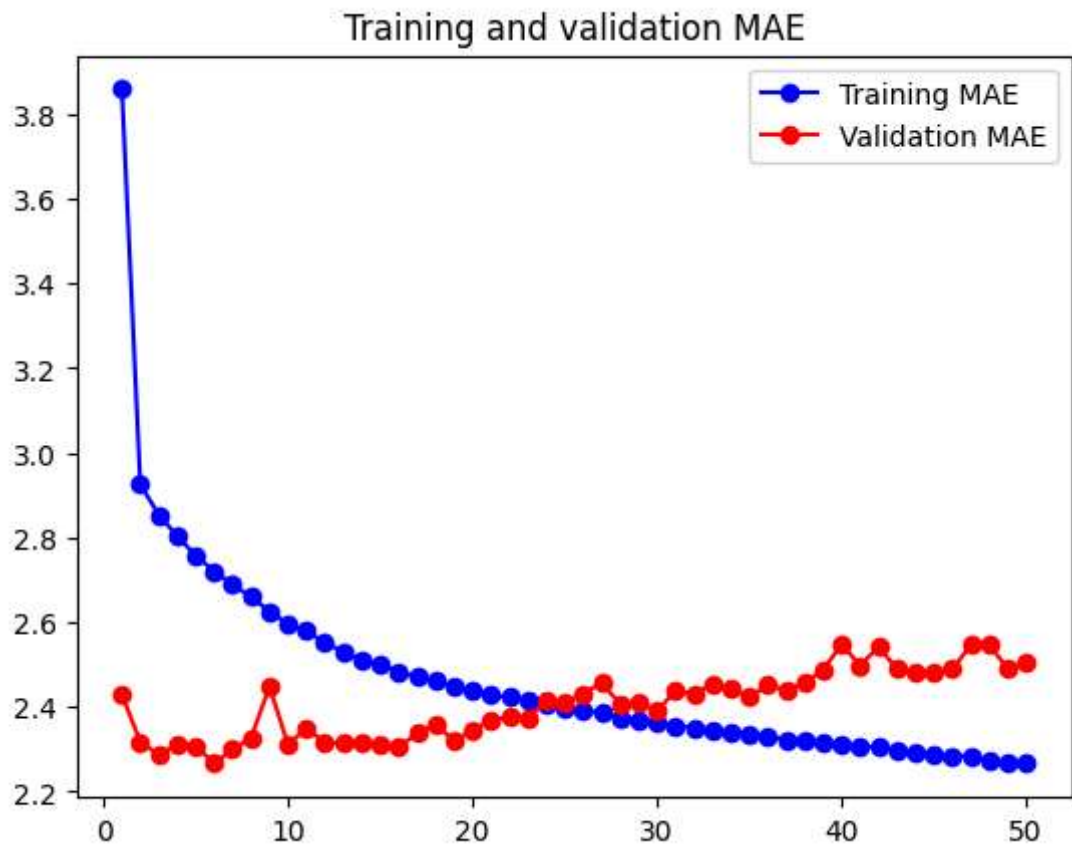
callbacks = [
    keras.callbacks.ModelCheckpoint("jena_stacked_gru_dropout.keras",
                                   save_best_only=True)
]
model.compile(optimizer="rmsprop", loss="mse", metrics=["mae"])
history = model.fit(train_dataset,
                    epochs=50,
                    validation_data=val_dataset,
                    callbacks=callbacks)
model = keras.models.load_model("jena_stacked_gru_dropout.keras")
print(f"Test MAE: {model.evaluate(test_dataset)[1]:.2f}")
```

Epoch 1/50
819/819 ————— **85s** 101ms/step - loss: 48.2295 - mae: 5.1603 - val_loss: 9.7948 - val_mae: 2.4285
Epoch 2/50
819/819 ————— **73s** 89ms/step - loss: 14.4825 - mae: 2.9491 - val_loss: 8.8977 - val_mae: 2.3180
Epoch 3/50
819/819 ————— **72s** 88ms/step - loss: 13.6211 - mae: 2.8588 - val_loss: 8.7229 - val_mae: 2.2861
Epoch 4/50
819/819 ————— **77s** 94ms/step - loss: 13.0917 - mae: 2.8066 - val_loss: 8.8149 - val_mae: 2.3118
Epoch 5/50
819/819 ————— **77s** 94ms/step - loss: 12.8082 - mae: 2.7694 - val_loss: 8.7981 - val_mae: 2.3083
Epoch 6/50
819/819 ————— **81s** 98ms/step - loss: 12.3513 - mae: 2.7262 - val_loss: 8.5098 - val_mae: 2.2661
Epoch 7/50
819/819 ————— **80s** 98ms/step - loss: 12.0634 - mae: 2.6946 - val_loss: 8.7443 - val_mae: 2.2993
Epoch 8/50
819/819 ————— **84s** 102ms/step - loss: 11.7224 - mae: 2.6588 - val_loss: 8.9206 - val_mae: 2.3267
Epoch 9/50
819/819 ————— **87s** 106ms/step - loss: 11.4522 - mae: 2.6271 - val_loss: 9.7219 - val_mae: 2.4476
Epoch 10/50
819/819 ————— **87s** 106ms/step - loss: 11.1253 - mae: 2.5926 - val_loss: 8.8299 - val_mae: 2.3112
Epoch 11/50
819/819 ————— **88s** 108ms/step - loss: 11.0118 - mae: 2.5798 - val_loss: 9.0745 - val_mae: 2.3497
Epoch 12/50
819/819 ————— **88s** 108ms/step - loss: 10.7900 - mae: 2.5560 - val_loss: 8.8308 - val_mae: 2.3157
Epoch 13/50
819/819 ————— **87s** 107ms/step - loss: 10.5753 - mae: 2.5330 - val_loss: 8.8605 - val_mae: 2.3146
Epoch 14/50
819/819 ————— **88s** 107ms/step - loss: 10.4265 - mae: 2.5120 - val_loss: 8.8274 - val_mae: 2.3134
Epoch 15/50
819/819 ————— **90s** 110ms/step - loss: 10.2985 - mae: 2.5003 - val_loss: 8.8643 - val_mae: 2.3111
Epoch 16/50
819/819 ————— **90s** 109ms/step - loss: 10.1626 - mae: 2.4816 - val_loss: 8.7701 - val_mae: 2.3046
Epoch 17/50
819/819 ————— **90s** 109ms/step - loss: 10.1005 - mae: 2.4767 - val_loss: 9.0023 - val_mae: 2.3375
Epoch 18/50
819/819 ————— **90s** 110ms/step - loss: 10.0132 - mae: 2.4636 - val_loss: 9.1111 - val_mae: 2.3589
Epoch 19/50
819/819 ————— **93s** 113ms/step - loss: 9.8480 - mae: 2.4446 - val_loss: 8.8728 - val_mae: 2.3198
Epoch 20/50
819/819 ————— **93s** 114ms/step - loss: 9.8090 - mae: 2.4406 - val_loss: 9.0430 - val_mae: 2.3440

Epoch 21/50
819/819 ————— **93s** 113ms/step - loss: 9.7294 - mae: 2.4261 - val_loss: 9.2510 - val_mae: 2.3694
Epoch 22/50
819/819 ————— **94s** 115ms/step - loss: 9.6663 - mae: 2.4251 - val_loss: 9.3073 - val_mae: 2.3754
Epoch 23/50
819/819 ————— **93s** 114ms/step - loss: 9.5729 - mae: 2.4123 - val_loss: 9.2855 - val_mae: 2.3722
Epoch 24/50
819/819 ————— **94s** 115ms/step - loss: 9.5540 - mae: 2.4086 - val_loss: 9.4978 - val_mae: 2.4157
Epoch 25/50
819/819 ————— **95s** 115ms/step - loss: 9.4744 - mae: 2.3965 - val_loss: 9.4486 - val_mae: 2.4109
Epoch 26/50
819/819 ————— **96s** 117ms/step - loss: 9.4343 - mae: 2.3911 - val_loss: 9.6370 - val_mae: 2.4307
Epoch 27/50
819/819 ————— **97s** 118ms/step - loss: 9.3712 - mae: 2.3860 - val_loss: 9.8188 - val_mae: 2.4578
Epoch 28/50
819/819 ————— **97s** 118ms/step - loss: 9.2721 - mae: 2.3743 - val_loss: 9.4484 - val_mae: 2.4062
Epoch 29/50
819/819 ————— **97s** 118ms/step - loss: 9.2415 - mae: 2.3705 - val_loss: 9.4339 - val_mae: 2.4096
Epoch 30/50
819/819 ————— **97s** 118ms/step - loss: 9.1925 - mae: 2.3633 - val_loss: 9.3635 - val_mae: 2.3904
Epoch 31/50
819/819 ————— **97s** 119ms/step - loss: 9.1528 - mae: 2.3537 - val_loss: 9.7668 - val_mae: 2.4382
Epoch 32/50
819/819 ————— **100s** 123ms/step - loss: 9.0890 - mae: 2.3495 - val_loss: 9.6646 - val_mae: 2.4296
Epoch 33/50
819/819 ————— **101s** 123ms/step - loss: 9.0291 - mae: 2.3448 - val_loss: 9.8626 - val_mae: 2.4548
Epoch 34/50
819/819 ————— **101s** 123ms/step - loss: 9.0394 - mae: 2.3441 - val_loss: 9.7345 - val_mae: 2.4429
Epoch 35/50
819/819 ————— **101s** 123ms/step - loss: 8.9918 - mae: 2.3361 - val_loss: 9.6223 - val_mae: 2.4236
Epoch 36/50
819/819 ————— **109s** 133ms/step - loss: 8.9659 - mae: 2.3332 - val_loss: 9.8781 - val_mae: 2.4512
Epoch 37/50
819/819 ————— **155s** 190ms/step - loss: 8.8732 - mae: 2.3210 - val_loss: 9.7639 - val_mae: 2.4407
Epoch 38/50
819/819 ————— **156s** 190ms/step - loss: 8.8178 - mae: 2.3183 - val_loss: 9.8607 - val_mae: 2.4562
Epoch 39/50
819/819 ————— **161s** 196ms/step - loss: 8.7802 - mae: 2.3114 - val_loss: 10.0199 - val_mae: 2.4879
Epoch 40/50
819/819 ————— **156s** 190ms/step - loss: 8.8140 - mae: 2.3123 - val_loss: 10.5371 - val_mae: 2.5478

Epoch 41/50
819/819  **154s** 188ms/step - loss: 8.7373 - mae: 2.3044 - val_loss: 10.1234 - val_mae: 2.4979
Epoch 42/50
819/819  **152s** 185ms/step - loss: 8.7612 - mae: 2.3087 - val_loss: 10.4227 - val_mae: 2.5417
Epoch 43/50
819/819  **156s** 191ms/step - loss: 8.6622 - mae: 2.2966 - val_loss: 10.1158 - val_mae: 2.4902
Epoch 44/50
819/819  **151s** 184ms/step - loss: 8.6556 - mae: 2.2954 - val_loss: 10.1010 - val_mae: 2.4797
Epoch 45/50
819/819  **151s** 185ms/step - loss: 8.5874 - mae: 2.2881 - val_loss: 10.0321 - val_mae: 2.4798
Epoch 46/50
819/819  **153s** 187ms/step - loss: 8.5288 - mae: 2.2790 - val_loss: 10.0972 - val_mae: 2.4889
Epoch 47/50
819/819  **152s** 186ms/step - loss: 8.5843 - mae: 2.2849 - val_loss: 10.5740 - val_mae: 2.5459
Epoch 48/50
819/819  **155s** 189ms/step - loss: 8.4886 - mae: 2.2686 - val_loss: 10.5940 - val_mae: 2.5476
Epoch 49/50
819/819  **155s** 190ms/step - loss: 8.4601 - mae: 2.2701 - val_loss: 10.1319 - val_mae: 2.4893
Epoch 50/50
819/819  **158s** 193ms/step - loss: 8.4190 - mae: 2.2633 - val_loss: 10.2858 - val_mae: 2.5045
405/405  **23s** 55ms/step - loss: 9.8610 - mae: 2.4640
Test MAE: 2.46

In [14]: `plot_mae(history)`

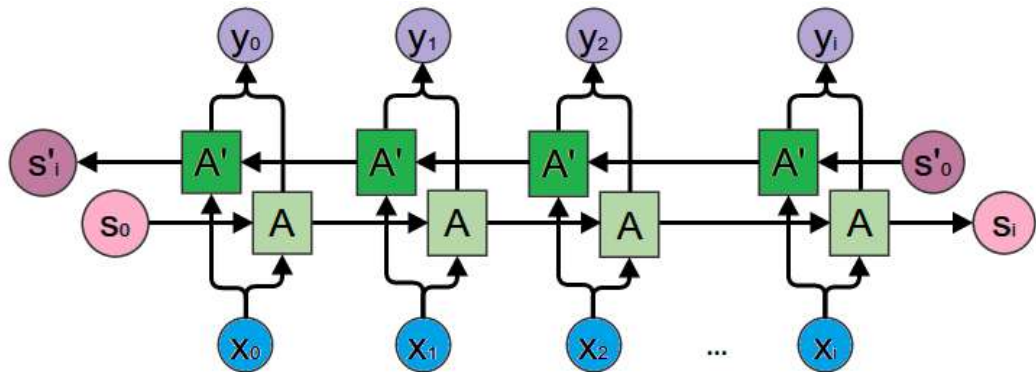


- We obtained the best results of all the recurrent versions.
- By increasing the capacity and expressive power of the network we made it more prone to overfitting.



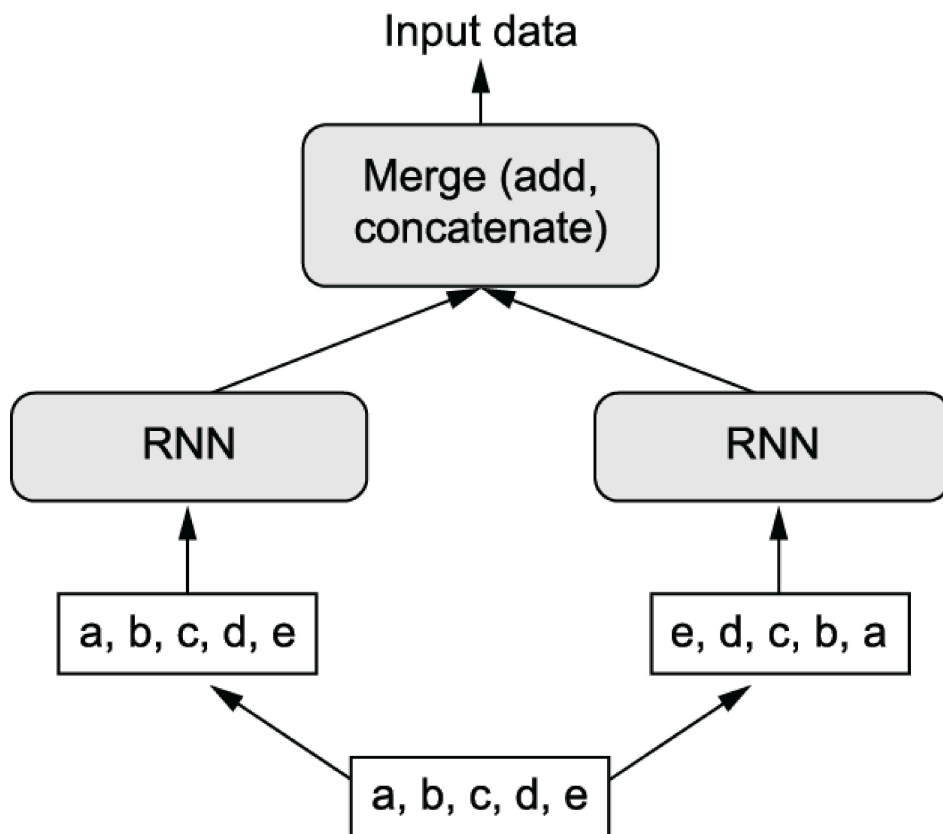
4.6.3. Bidirectional RNNs

- RNNs are notably order-dependent: they process the timesteps of their input sequences in order.
- This is precisely the reason they perform well on problems where order is meaningful, such as the temperature-forecasting problem.
- Patterns that do not follow the increasing sequence order may be overlooked by a unidirectional RNN.
- A **bidirectional RNN** uses two regular RNNs (LSTM or GRU layers), each of which processes the input sequence in one direction (chronologically and antichronologically), and then merges their representations.



Advantages

- A bidirectional RNN exploits looking at its input sequence both ways, obtaining potentially richer representations and capturing patterns that may have been missed by the chronological-order version alone.
- They are not very useful in problems where the chronological processing is important to the success of the approach (time series forecasting).
- They are **frequently used in natural language processing**, in which chronological and antichronological order are meaningful.



Bidirectional RNNs in Keras

- To instantiate a bidirectional RNN in Keras, we use the `Bidirectional` layer, which takes as its first argument a recurrent layer instance.
- Bidirectional creates a second, separate instance of this recurrent layer and uses one instance for processing the input sequences in chronological order and the other instance for processing the input sequences in reversed order.
- Let's try it on our temperature-forecasting task.

```

In [15]: inputs = keras.Input(shape=(sequence_length, raw_data.shape[-1]))
x = layers.Bidirectional(layers.LSTM(16))(inputs)
outputs = layers.Dense(1)(x)
model = keras.Model(inputs, outputs)

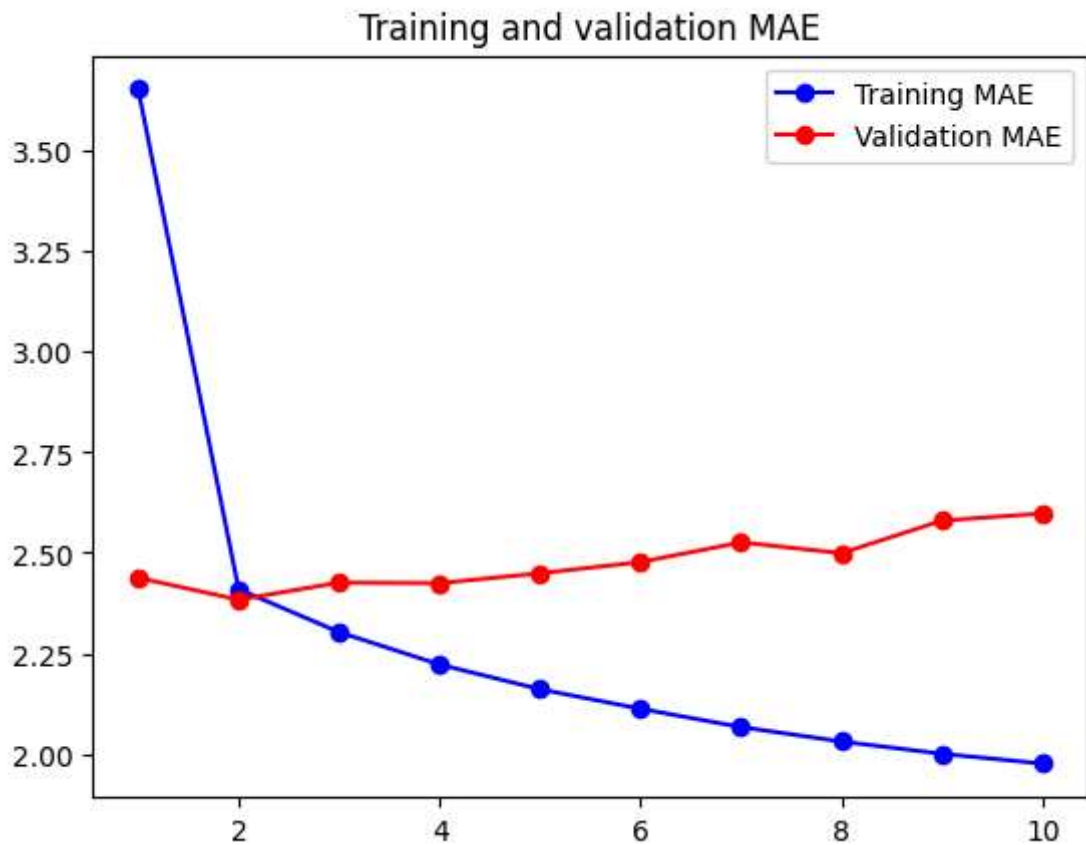
callbacks = [
    keras.callbacks.ModelCheckpoint("jena_bidirectional_lstm.keras",
                                   save_best_only=True)
]

model.compile(optimizer="rmsprop", loss="mse", metrics=["mae"])
history = model.fit(train_dataset,
                    epochs=10,
                    validation_data=val_dataset,
                    callbacks=callbacks)
model = keras.models.load_model("jena_bidirectional_lstm.keras")
print(f"Test MAE: {model.evaluate(test_dataset)[1]:.2f}")

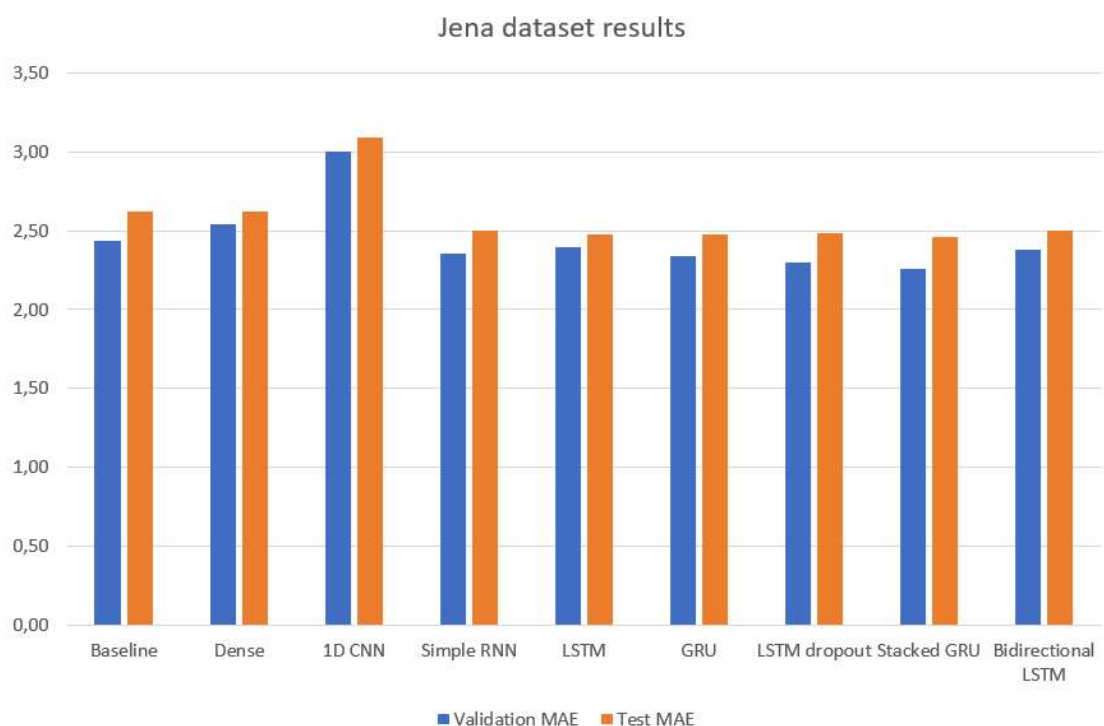
```

Epoch 1/10
819/819  **73s** 87ms/step - loss: 53.0816 - mae: 5.3205 - val_loss: 9.9264 - val_mae: 2.4380
Epoch 2/10
819/819  **70s** 85ms/step - loss: 9.8255 - mae: 2.4424 - val_loss: 9.5054 - val_mae: 2.3834
Epoch 3/10
819/819  **70s** 85ms/step - loss: 8.9366 - mae: 2.3244 - val_loss: 9.8513 - val_mae: 2.4260
Epoch 4/10
819/819  **69s** 84ms/step - loss: 8.3418 - mae: 2.2413 - val_loss: 9.8263 - val_mae: 2.4246
Epoch 5/10
819/819  **71s** 87ms/step - loss: 7.8238 - mae: 2.1729 - val_loss: 10.0060 - val_mae: 2.4492
Epoch 6/10
819/819  **70s** 86ms/step - loss: 7.4407 - mae: 2.1212 - val_loss: 10.3270 - val_mae: 2.4771
Epoch 7/10
819/819  **70s** 85ms/step - loss: 7.1041 - mae: 2.0761 - val_loss: 10.6730 - val_mae: 2.5261
Epoch 8/10
819/819  **70s** 85ms/step - loss: 6.8490 - mae: 2.0393 - val_loss: 10.3852 - val_mae: 2.4992
Epoch 9/10
819/819  **70s** 85ms/step - loss: 6.6163 - mae: 2.0057 - val_loss: 11.1372 - val_mae: 2.5801
Epoch 10/10
819/819  **70s** 85ms/step - loss: 6.4472 - mae: 1.9841 - val_loss: 11.3925 - val_mae: 2.5979
405/405  **16s** 38ms/step - loss: 10.1352 - mae: 2.4989
Test MAE: 2.50

In [16]: `plot_mae(history)`



- It can be seen that it doesn't perform very well: the results are the worst among all recurring networks.
- It's easy to understand why: all the predictive capacity must come from the chronological half of the network, because the antichronological half is known to be severely underperforming on this task.
- At the same time, the presence of the antichronological half doubles the network's capacity and causes it to start overfitting much earlier.



Bidirectional LSTM on IMDB

- Let's try the bidirectional architecture with an example of that we already know from the regularization unit: the IMDB dataset.
- This is a dataset of 25,000 movies reviews from IMDB, labeled by sentiment (positive/negative) that we want to predict.

Setup

```
In [17]: import numpy as np
import keras
from keras import layers

max_features = 20000 # Only consider the top 20k words
maxlen = 200 # Only consider the first 200 words of each movie review
```

Load the IMDB movie review sentiment data

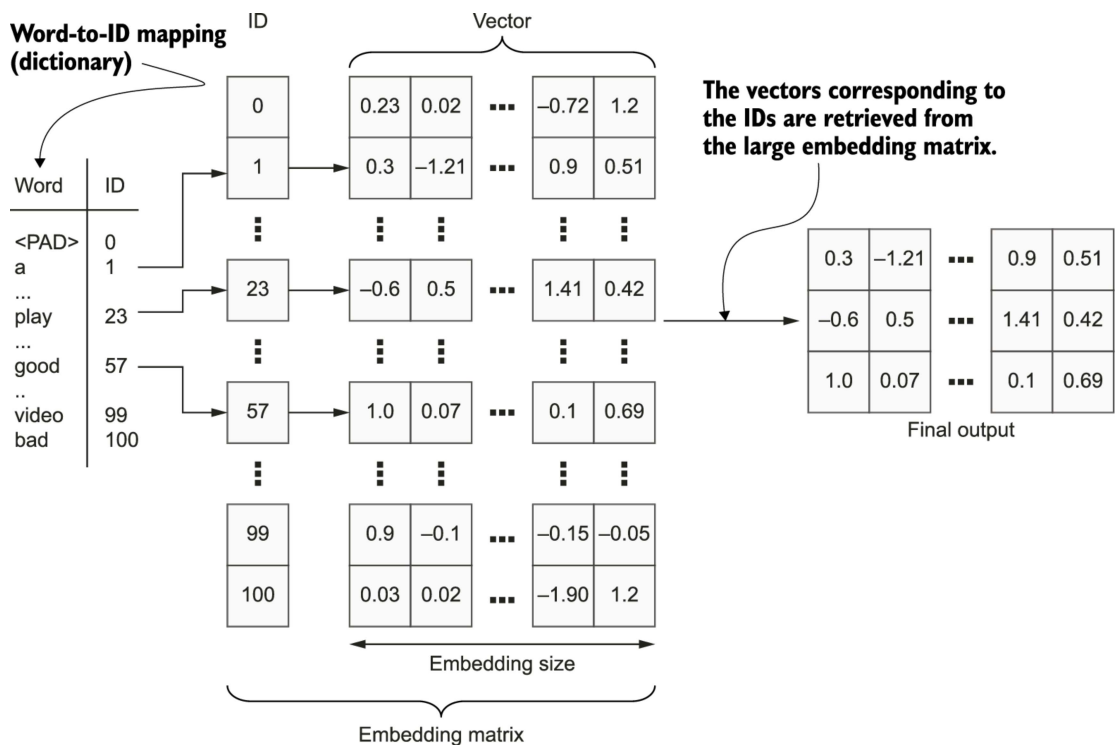
- With the `pad_sequences` function we pad all the sequences to the same length, which is determined by the `maxlen` parameter.

```
In [18]: (x_train, y_train), (x_val, y_val) = keras.datasets.imdb.load_data(
        num_words=max_features
    )
print(len(x_train), "Training sequences")
print(len(x_val), "Validation sequences")
x_train = keras.preprocessing.sequence.pad_sequences(x_train, maxlen=maxlen)
x_val = keras.preprocessing.sequence.pad_sequences(x_val, maxlen=maxlen)
```

```
25000 Training sequences
25000 Validation sequences
```

Word embeddings

- We are not using multi-hot encoding, such as in the regularization unit.
- We will use an **word embeddings** that converts each integer representing a word into a vector.
- An embedding is a dense vector of floating point values (the length of the vector is a parameter you specify).
- Word embeddings give us a way to use an efficient, dense representation in which similar words have a similar encoding.



- Instead of specifying the values for the embedding manually, **they are trainable parameters** (weights learned by the model during training, in the same way a model learns weights for a dense layer).
- It is common to see word embeddings that are 8-dimensional (for small datasets), up to 1024-dimensions when working with large datasets. We are using a 128-dimensional vector.
- A higher dimensional embedding can capture fine-grained relationships between words, but takes more data to learn.
- Embeddings are done in **Keras** using the `Embedding` layer (https://keras.io/api/layers/core_layers/embedding/).

Build the model

- We build the model as a bidirectional network with two stacked LSTM layers.
- Since this complex model is prone to overfit we try to avoid it by using dropout and recurrent dropout in the LSTM cells, and we add an additional dropout layer before the output dense layer.

```
In [19]: # Input for variable-length sequences of integers
inputs = keras.Input(shape=(None,), dtype="int32")

# Embed each integer in a 128-dimensional vector
x = layers.Embedding(max_features, 128)(inputs)

# Add 2 bidirectional LSTMs
x = layers.Bidirectional(layers.LSTM(64, dropout=0.2, recurrent_dropout=0.2, re
x = layers.Bidirectional(layers.LSTM(64, dropout=0.2, recurrent_dropout=0.2))(x

# Add a classifier
x = layers.Dropout(0.5)(x)
outputs = layers.Dense(1, activation="sigmoid")(x)
```

```
model = keras.Model(inputs, outputs)
model.summary()
```

Model: "functional_5"

Layer (type)	Output Shape	Param #
input_layer_5 (InputLayer)	(None, None)	0
embedding (Embedding)	(None, None, 128)	2,560,000
bidirectional_1 (Bidirectional)	(None, None, 128)	98,816
bidirectional_2 (Bidirectional)	(None, 128)	98,816
dropout_2 (Dropout)	(None, 128)	0
dense_5 (Dense)	(None, 1)	129



Total params: 2,757,761 (10.52 MB)

Trainable params: 2,757,761 (10.52 MB)

Non-trainable params: 0 (0.00 B)

Train and evaluate the model

```
In [20]: model.compile("adam", "binary_crossentropy", metrics=["accuracy"])
history = model.fit(x_train, y_train, batch_size=256, epochs=10, validation_data=
```

Epoch 1/10
98/98  **302s** 3s/step - accuracy: 0.6356 - loss: 0.6088 - val_accuracy: 0.8470 - val_loss: 0.3630
Epoch 2/10
98/98  **282s** 3s/step - accuracy: 0.8656 - loss: 0.3409 - val_accuracy: 0.8560 - val_loss: 0.3456
Epoch 3/10
98/98  **283s** 3s/step - accuracy: 0.9208 - loss: 0.2212 - val_accuracy: 0.8541 - val_loss: 0.3580
Epoch 4/10
98/98  **281s** 3s/step - accuracy: 0.9410 - loss: 0.1704 - val_accuracy: 0.8500 - val_loss: 0.3834
Epoch 5/10
98/98  **282s** 3s/step - accuracy: 0.9566 - loss: 0.1287 - val_accuracy: 0.8434 - val_loss: 0.4659
Epoch 6/10
98/98  **279s** 3s/step - accuracy: 0.9683 - loss: 0.0985 - val_accuracy: 0.8487 - val_loss: 0.4652
Epoch 7/10
98/98  **277s** 3s/step - accuracy: 0.9667 - loss: 0.1026 - val_accuracy: 0.8384 - val_loss: 0.4611
Epoch 8/10
98/98  **277s** 3s/step - accuracy: 0.9599 - loss: 0.1185 - val_accuracy: 0.8415 - val_loss: 0.5079
Epoch 9/10
98/98  **279s** 3s/step - accuracy: 0.9684 - loss: 0.0933 - val_accuracy: 0.8459 - val_loss: 0.5319
Epoch 10/10
98/98  **279s** 3s/step - accuracy: 0.9771 - loss: 0.0715 - val_accuracy: 0.8492 - val_loss: 0.5410

```
In [21]: import matplotlib.pyplot as plt

# Training and validation loss
history_dict = history.history
loss_values = history_dict['loss']
val_loss_values = history_dict['val_loss']

epochs = range(1, len(loss_values) + 1)

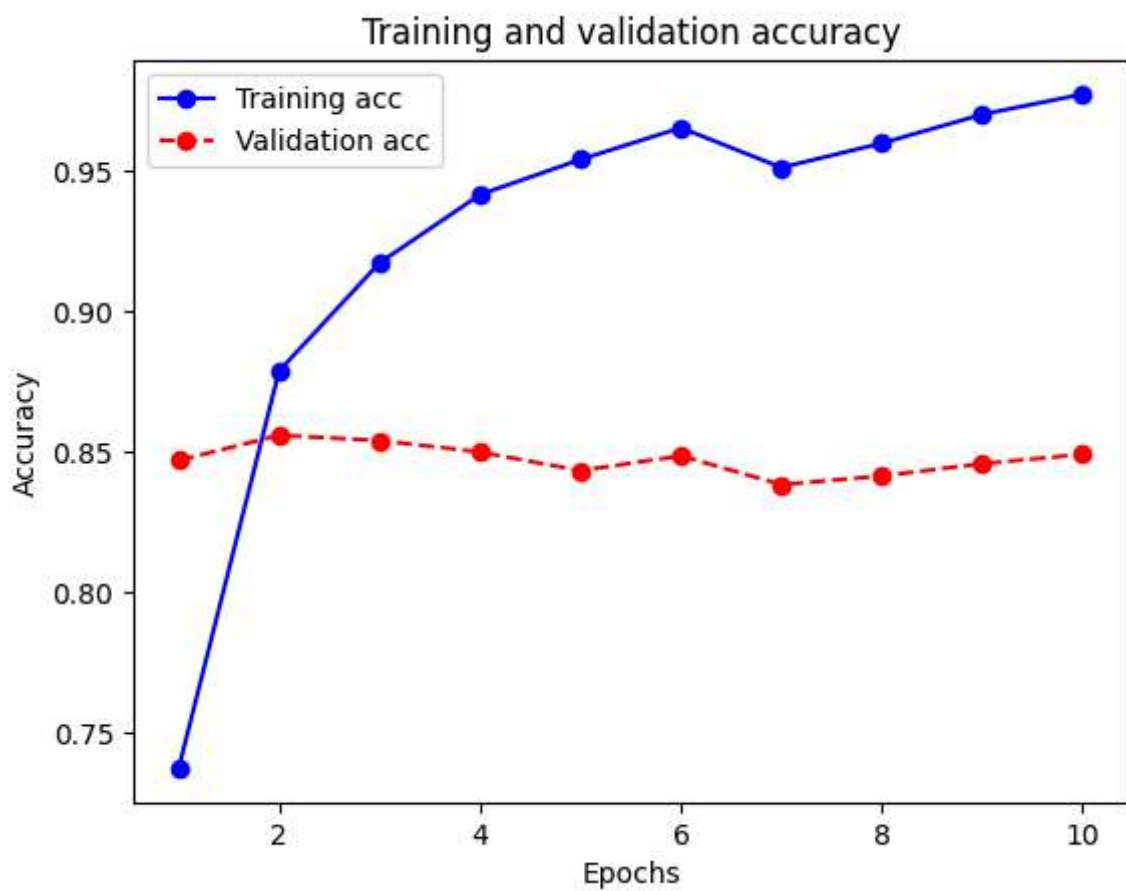
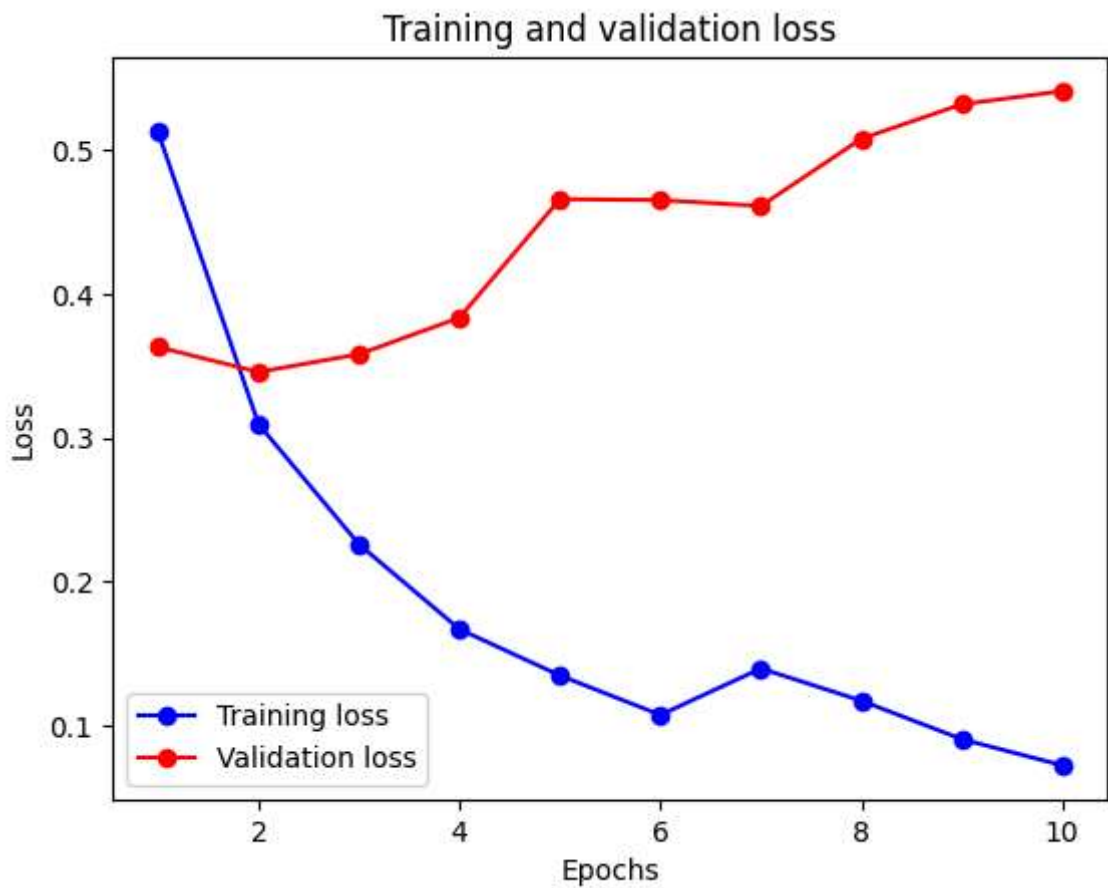
plt.plot(epochs, loss_values, 'b-o', label='Training loss')
plt.plot(epochs, val_loss_values, 'r-o', label='Validation loss')

plt.title('Training and validation loss')
plt.xlabel('Epochs')
plt.ylabel('Loss')
plt.legend()
plt.show()

# Training and validation accuracy
plt.clf()
acc = history_dict['accuracy']
val_acc = history_dict['val_accuracy']

plt.plot(epochs, acc, 'b-o', label='Training acc')
plt.plot(epochs, val_acc, 'r--o', label='Validation acc')
plt.title('Training and validation accuracy')
plt.xlabel('Epochs')
plt.ylabel('Accuracy')
plt.legend()
```

```
plt.show()
```



- The results are as good as when we used dense networks over this problem.

- We also faced, as the previous time, important overfitting problems.